Building a Blog with Vite + React, Strapi, and Tailwind CSS

Tyler Sanderson

Wilmington University

Experiential Learning in CSC (22672.SEM)

Charles A. Bachman, PhD MBA PMP

May 4, 2025

Abstract

This project involves the development of a dynamic personal blog/portfolio website designed to showcase computer science skills and projects. The website, developed using Vite, React, TailwindCSS, and Strapi, serves both as a portfolio for technical projects and as a platform for sharing knowledge through blog posts. The project is executed in a hybrid Agile framework, spanning 14 weeks and broken down into 6 two-week sprints, ensuring iterative development, continuous feedback, and flexibility to meet deadlines. The development process involves technical architecture discussions, including high-level user-flow diagrams, wireframes, entity relationship diagrams, and relational schemas, addressing both functional and non-functional requirements.

Key technical considerations include the use of Git, GitHub, JSX syntax, ReactRouter, Apollo, and GraphQL for integrating front-end and back-end components. Challenges during development included route management, integrating social media features, improving search functionality with Meilisearch, and resolving performance and accessibility issues. Testing phases involved integration and E2E testing, addressing issues like content wrapping and code block rendering. Despite these challenges, the platform successfully met most business requirements and demonstrated progress in responsive design and core functionalities.

This project also evaluated the scalability of the platform, identifying potential limitations and proposing a future migration to a virtual private server (VPS) to handle increased traffic. Security and deployment challenges during this transition were mitigated using SSH keys, UFW, and Crowdsec, with ongoing research into Docker Swarm and GitHub Actions for CI/CD. While the platform is partially deployed on the VPS, additional documentation and support options are still being explored. This comprehensive development approach ensures that the platform is not

only functional and user-friendly, but also scalable and secure for future expansion!

Building a Blog with Vite + React, Strapi, and Tailwind CSS

Project Overview:

A portfolio is integral to any professional's working career. Being able to properly manage and document projects is key to properly communicating one's skills and achievements to other professionals. This project aims to develop a blog-style portfolio platform that can easily be maintained over time and can serve as extra documentation to our projects and portfolios as we learn and continue to grow. Additionally, this project will serve as a dynamic and interactive showcase of the skills and expertise I've learned throughout my computer science program.

Using the Agile methodology, separated into 7 iterative sprints. Each sprint lasts 2-weeks, totalling a 14-week project. Agile's iterative approach encourages flexibility and continuous feedback, allowing me to update and refine the website as time progresses (GeeksforGeeks & prachipaliwal1207, 2024). This project overview outlines the key phases, tasks, and milestones to complete said project.

Sprint 1 will define the project scope and high-level features for the website. We'll work on determining the minimum viable product for the project, determining the tools and technologies to be used (such as React, MongoDB, next.js, etc) (Limosyn, 2021), wireframing and general design of the website, and we'll set-up our coding environment and version control for the project. At the end of sprint 1 (Feb 2), we should have a backlog of tasks and features, initial wireframes/designs, and a chosen tech stack to work with.

Sprint 2 will create the basic structure of the website. We'll set-up the project repository and folder structure, set-up the main pages (Login, Home, About, Blog, etc.), add navigation, and we'll set-up the development environment (server and build tools). At the end of sprint 2 (Feb 16), we should have a basic functional website to structure and serve as placeholders for our

project.

Sprint 3 will focus on user-design and experience. We'll implement our wireframes and designs into the layout, make the website more mobile-friendly, and test the users flow and navigation feels intuitive. This sprint will be highly iterative. At the end of sprint 3 (Mar 2), the website should have a responsive design and consistent layout.

Sprint 4 will focus on setting up the blogging system and content management system. We'll implement the ability to create, edit, and display blog posts; as well as sort and filter them based on applicable tags. At the end of sprint 5 (Mar 23), we should have a fully functional blog system and one to two sample posts!

Sprint 5 we'll focus on adding an 'about me' page and filling in our portfolio into the blogging system. The portfolio should list key projects we've worked on and provide further detail and insights, such as descriptions, images, github links, or live-demonstrations. At the end of sprint 5 (Apr 6), we should have at least 3-4 populated entries showcasing my work.

Sprint 6 will focus on adding additional features and polish to the website. We'll implement a contact form, email integration, and we'll add a comment section to blog posts. At the end of sprint 6(Apr 20) we should have a fully functional contact form with email integration and a fully functional comment section to each blog post.

Sprint 7 will focus on further testing, bug-fixing, and deployment. We'll conduct further usability testing, fix any known issues or bugs. We'll optimize the performance of the website through image compression techniques and consolidation techniques, like minifyJS, to improve the performance of our website. Finally we'll deploy the website to a live server. At the end of sprint 7 (May 4), we should have a fully functional blog-style website to house our portfolios and it should be accessible online!

Additionally, the agile process encourages continuous development. Post-launch, we'll be able to gather further feedback and gain insight into the website's performance using web analytic tools. We'll also be able to add to our portfolios through our content management system. While not an official sprint in this project, continuous refinement of features and content is key to the successful longevity of any product. At the end of this project, we should have a fully functional blog-style portfolio website that is accessible online; a user- friendly blogging platform for sharing knowledge, a responsive website that adapts to various devices and screen sizes, and comprehensive project documentation detailing the development of the said website.

To conclude this overview, this project's goal is to create a personal blog-style portfolio website that not only demonstrates my computer skills, but also serves as a platform for ongoing learning and knowledge sharing. Using the Agile approach gives the project structure while also leaving room for continuous iteration and changing demands that may fall into the project's scope as the project progresses. Naturally, feedback and iteration will shape the website's development ensuring the website meets functional and aesthetic requirements.

Planning: Building a Blog with Vite + React, Strapi, and Tailwind CSS

A portfolio is integral to any professional's working career. Being able to properly manage and document projects is key to properly communicating one's skills and achievements to other professionals. This project aims to develop a blog portfolio platform that can easily be maintained over time and can serve as extra documentation to our projects and portfolios as we learn and continue to grow. By utilising a modern tech stack - Vite+React, TailwindCSS, and Strapi, this project will serve as a dynamic and interactive showcase of the skills and expertise I've learned throughout my computer science program. This project should deliver a fully functioning blog that includes a front-end, using Vite+React and TailwindCSS, and a back-end

content management system, using Strapi. This plan serves as the guide to successfully achieving project completion within the deadlines requested.

We will be using Vite to start our initial React build, instead of create-react-app, as we normally would. Vite has ECMAScript module support, or ES module support, which, for our purposes, means it reduces the amount of code needed at any one time! This is more efficient and speeds up our set-up time, as well as it provides better support for testing in the browser (Amal, 2024). TailwindCSS, will allow us to quickly and effectively design our front-end within our mark-up (Tailwind).

While initial ideation of this project included creating our own content management system, further research from this initial planning sprint found the scope of such a project may not necessarily be feasible due to the projects allotted deadlines. Building a proper content management system, or CMS, would actually involve creating a content management application, or CMA, as well as a content delivery application, or CDA (Finn & Downie, 2024). Since I do not have the desire to build a business in content management software, nor do I have specific regulatory demands that I must meet, further research was done into Strapi (Palas, 2017). Strapi is a headless open-source CMS, which means it handles the content management and administration, the back-end, while also being part of a larger open-source community, ensuring long-term support and additional functionality. Later on, plugins for Strapi could help us implement comments, and potentially other features later down the line (Strapi).

Finally, we'll be using a hybrid Agile approach for this project. We'll be combining Agile's principles of flexibility (GeeksforGeeks & prachipaliwal1207, 2024) with structured waterfall-like phases in order to more appropriately meet this class's scheduled deadlines and deliverables. To adapt Agile to this project's timeline, we will separate each of the phases into

sprints, allowing for a planning and analysis phase at the start of each iteration, as well as reflection at the end of each sprint. By leveraging these classes assignments and deliverables, we can guarantee continuous feedback from our peers and instructor throughout the life of the project.

Using this hybrid-Agile approach, we've split the project into 6 sprints. Each sprint lasts 2-weeks, totalling a 14-week project. This project outlines the key phases, tasks, and milestones to complete said project.

Sprint 1 - The Planning Phase (Deadline: Feb 2)

Sprint 1, which is the cumulation of this project plan, set-out to define the project and its scope. This sprint included allocating time for research and resulted in key deliverables, such as the project's initial outline, a confirmed tech stack, and an updated project plan that more closely falls within the project's requested deadlines.

Sprint 2 - Analysis Phase (Deadline: Feb 16)

Sprint 2 will start with planning time to gather and prioritize requirements, user stories, and any feedback received. In sprint 2, we will further solidify the requirements of the website and start structuring our blog, which should also include setting-up the packages dependencies needed in VSCode, our chosen development environment. Key deliverables will include a more comprehensive analysis of the technical requirements and structure needed to complete the project, including stakeholders, as well as rough schema and wireframes for the website, and a backlog of tasks to complete.

Sprint 3 - Design Phase (Deadline: Mar 2)

In sprint 3, will pick up on sprint 2 by finalising the UX design for the website and mapping the API schema for Strapi. We'll start by planning time to design said user interface

and backend schema; and key deliverables will include more comprehensive wireframes, mock-ups, and schema relating Strapi's back-end to our front-end, as well as a reflection detailing these events. If time permits, we can start adding the initial structure and pages to our website.

Sprint 4 - Development Phase (Deadline: Mar 23)

Sprint 4 will focus strictly on development. We'll review our backlog and set time aside to code and implement our designs into the tech stack. First we'll finalise our front-end design, and then we'll work on integrating our API calls using GraphQL instead of REST, due to its simplicity, performance advantages, and integration with our chosen front-end tech stack, Vite+React (Bajgain, 2023). Key deliverables will include a basic functional blog that includes front-end rendering and back-end API calls, as well as a reflection of said sprint and its activities.

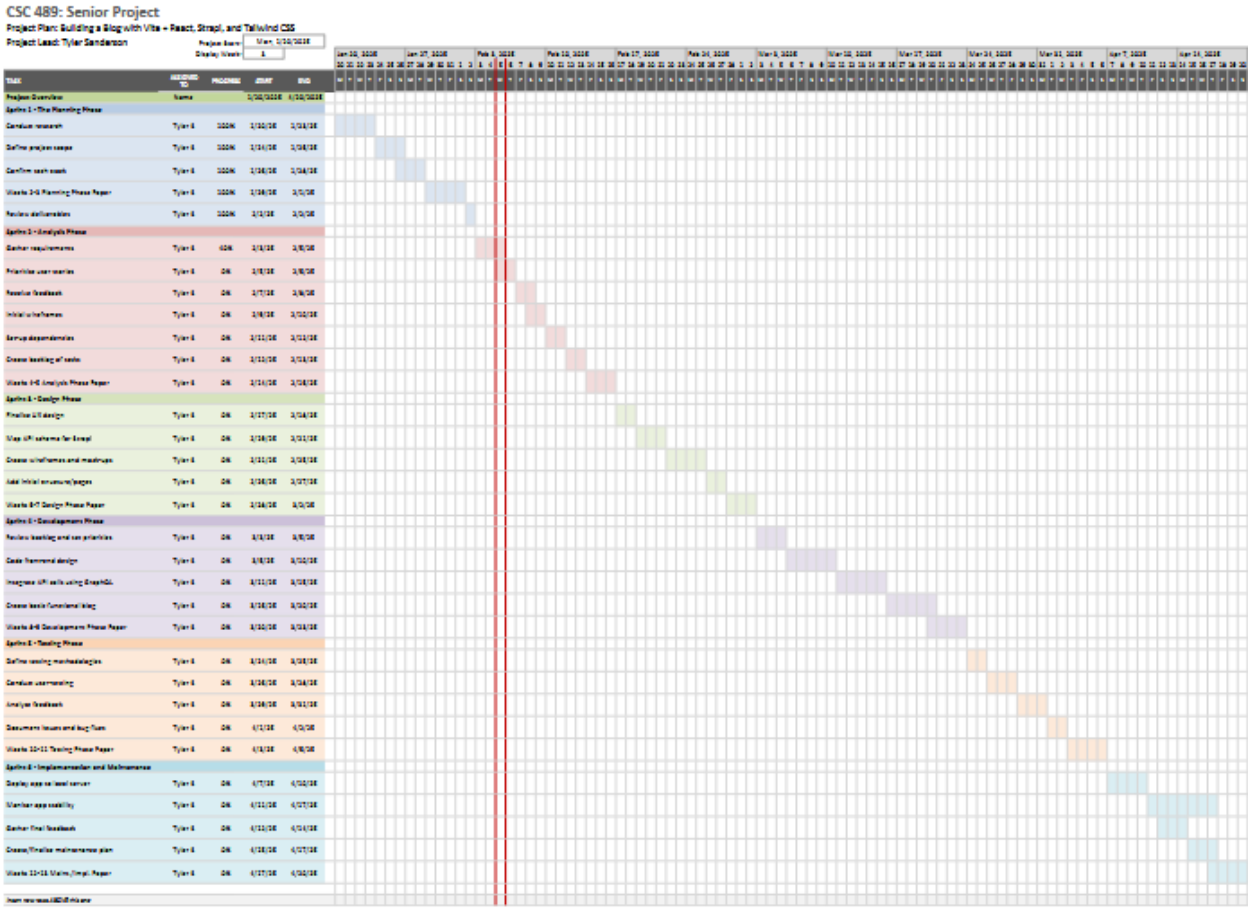
Sprint 5 - Testing Phase (Deadline: Apr 6)

Sprint 5 will run our app through its paces. We'll plan and define testing methodologies and set aside time to apply them against the project. We'll utilize user-testing with peers and family and gather feedback based on features and we'll analyse feedback to document potential issues, bug fixes, and performance optimizations. Key deliverables will include a reflection, documentation, and discussion of the sprints activities.

Sprint 6 - Implementation and Maintenance (Deadline: Apr 20)

Sprint 6 will focus on finally implementing the project to our local server, either using Nginx proxy manager and Cloudflare, or Cloudflare Tunnel to get our assets served to the web for deployment! During this phase, we'll set aside time to deploy the app and ensure it's stable operation. We'll monitor and gather feedback from peers and family; and we'll set aside time to assess the project's completion and that desired business goals have been met by the project. Key deliverables for this sprint will include a reflection of said activities, a deployed/running blog

app, and a maintenance plan for how the system may scale, or not scale, as well as how it will generally be administered in production moving forward.



Additionally, this hybrid agile approach encourages continuous development. Post-launch, we'll be able to gather further feedback and gain insight into the website's performance using web analytic tools, such as Cloudflare. We'll also be able to add to our portfolios through our content management system, Strapi. While not an official sprint in this project, continuous refinement of features and content is key to the successful longevity of any product. At the end of this project, we should have a fully functional blog-style portfolio website that is accessible online; a user- friendly blogging platform for sharing knowledge, a responsive website that adapts to various devices and screen sizes, and comprehensive project documentation detailing the development of the said website.

To conclude our planning phase, utilising Vite+React, TailwindCSS, and Strapi, this project's goal is to create a personal blog-style portfolio website that not only demonstrates my computer skills, but also serves as a platform for ongoing learning and knowledge sharing on the internet. Using a hybrid Agile approach gives the project reasonable structure while also leaving room for continuous iteration and changing demands that may fall into the project's scope as the project progresses. Naturally, feedback and iteration will shape the website's development ensuring the website meets functional and aesthetic requirements.

Analysis Phase: Building a Blog with Vite + React, Strapi, and Tailwind CSS

Utilising a modern tech stack - Vite+React, TailwindCSS, and Strapi, this project aims to develop a blog platform that can be easily maintained over time and serve as extra documentation to users' projects and portfolios as they learn and continue to grow; additionally, serving as a dynamic and interactive showcase of the skills and expertise I've learned throughout my computer science program. The future target audience for whom may be developers, students, or professionals looking for an interactive and visually appealing way to document their work and learning, but will be served locally with smaller ambitions of 2 home users for the time being, until later sprints are iterated to for a larger VPS deployment. This analysis serves to define the various stakeholders who may be interested in this project; as well as it serves to outline the various features/functions and non-functional/technical requirements needed to achieve said demand; as well as an analysis of the potential business implications such a product could entail!

The current project is only slated to serve two user profiles to a small subset of colleagues, friends and family, and potential employers; but will be set up locally in a way that is easily deployable to a larger VPS, or Virtual Private Server, in the future! This means the current

potential daily load on the system may be small, maybe a max of 100 requests for the day; but as my skills, hopefully money, and community grow, I may want to move the system to a larger VPS system. I could do this to grow the community I hope to serve, empower other creators to be hosted or serve themselves, and potentially help people get employed or even make a potential business venture off the project! By building our site with Vite+React, TailwindCSS, Strapi, and NginX Proxy Manager on a linux container, we ensure the project is easily replicated to a larger VPS system and scales well to future stakeholders! The following analysis will define end users, collaborative IT specialists, and business requirements in relation to current and potential future stakeholders respectively.

Again, while the end users are currently a small subset of people, previously mentioned, the future stakeholders may include a growing community of collaborators and professionals, employers, or may even serve as a platform in of itself to a larger audience of collaborative professionals outside the current scope of this project. Their goals may include being able to document their works and studies through a blog-style format using clear controls, inputs, formats, text, images, embedded video, and code snippets! Strapi's locally hosted back-end will luckily handle all of this for us, to be later detailed in this document. These needs greatly shaped the decision to use the chosen tech stack, Vite + React, Strapi, and Tailwind CSS, which we will now refer to as VRTS. The site needs to be self-contained for future deployments, so we chose to use an Ubuntu linux container, on Proxmox, as our host for this project. This ensures an easy and compatible deployment to a larger VPS down the line, since the same technologies can be used there. By using Vite+React, we ensure a speedy and reactive deployment to the potential users browsers! By using TailwindCSS, we ensure a clear design philosophy to ensure clear reactive inputs to users, and with Strapi we ensure clear reactive controls to content creators and ensure

scalability and security in that it is open-source and is frequently updated.

The IT specialists and collaborators involved in collecting said data included myself, as well as friends and colleagues who have also expressed interest in being able to more affordably and efficiently host their works and interest; and have provided me with their user-stories to build a set of functional/non-functional requirements for a platform that serves their needs and portfolios!

As part of the analysis phase, I reflected on the previous iteration by researching Strapi's API calls and setting up the environment with Vite on my local server within an instance of Visual Studio Code on an Ubuntu Linux container on Proxmox within my own locally hosted server; then, I spent the next few days talking to friends, colleagues, and other professionals to see what they've been struggling to show off, either due to limitations in their own time and budgets. Concisely, here's a few that stood out:

"I'd like to be able to just drop everything I'm working on for a particular post or topic into one thing, like code-snippets and various file types together!"

"I wanna be able to feature whatever I want at the top!"

"LinkedIn is great, but it's hard to format things how you want sometimes, especially if you aren't paying for their LinkedIn premium. Sites like Squarespace costs a bit much for just serving my portfolio and other solutions like Pinterest, Wordpress, and they either lack the features I want, or the costs."

- (Sanderson & Superior)

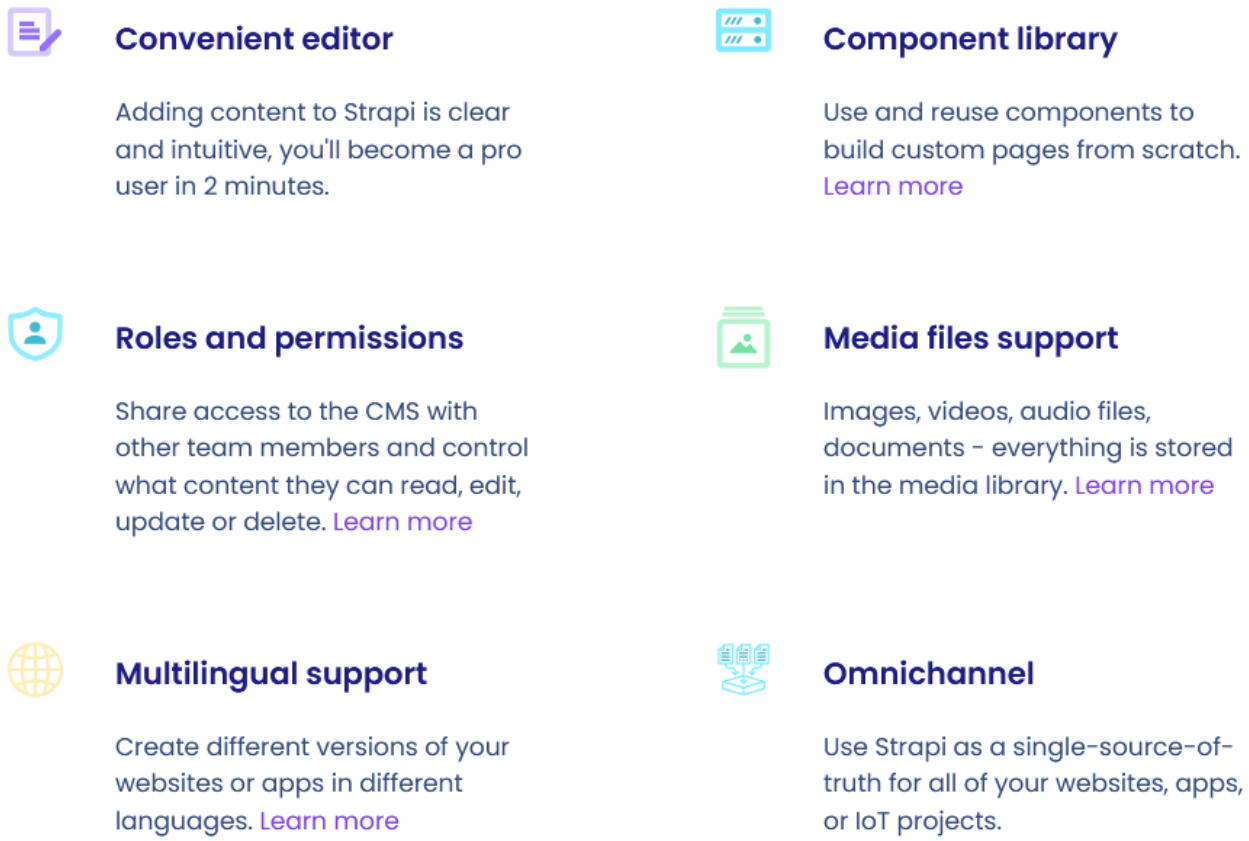
"I'd like to be able to just hop on and write, similar to 'archive of your own', but maybe with some more controls?"

- (Sanderson & Partner)

"Yeah, I'd just want to be able to show off all my certificates and portfolio[…]"

- (Sanderson & Family Member)

Nextly, some time was spent monitoring current solutions that support developer and general portfolios to see what was successful already. From the content management side of this project, Strapi handles the bulk of users requests, and then some, and scales per their open-source license agreement.

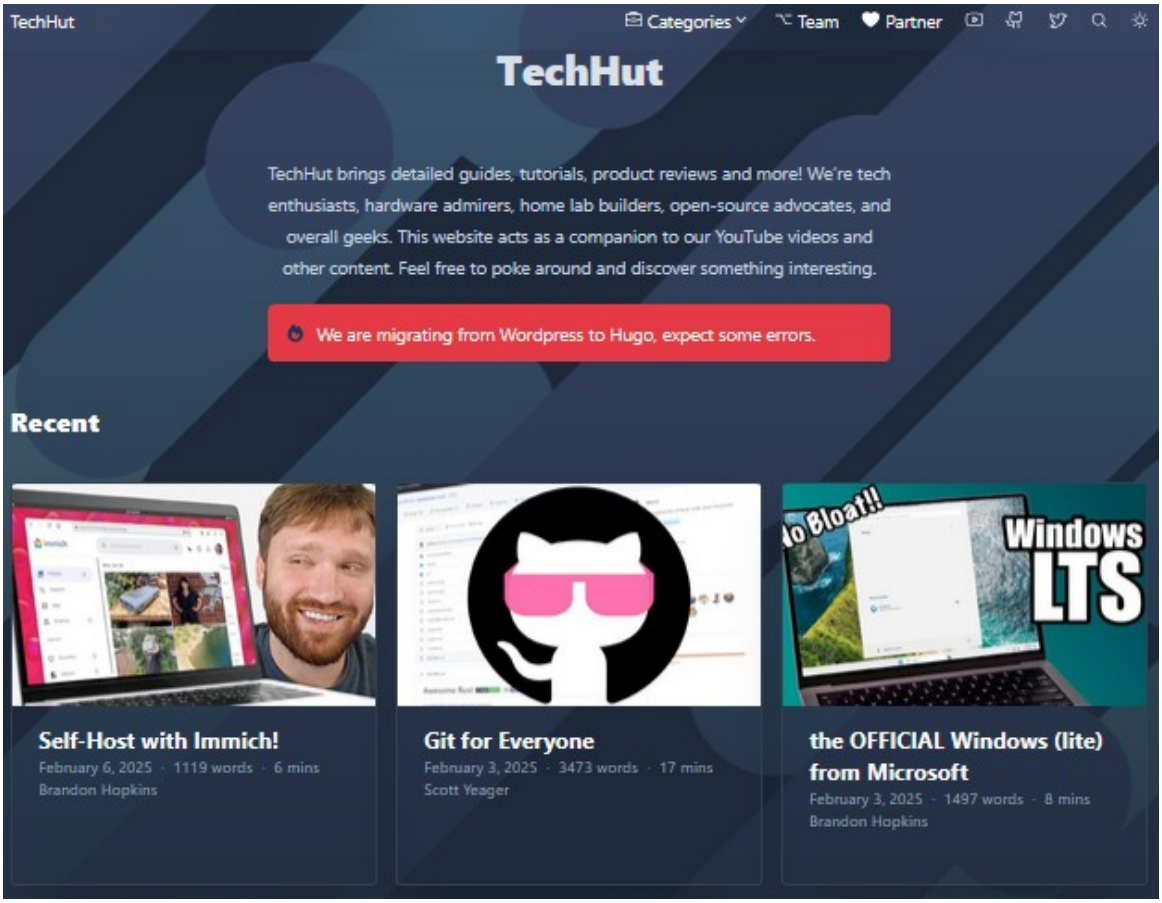


(Strapi CMS for content teams)

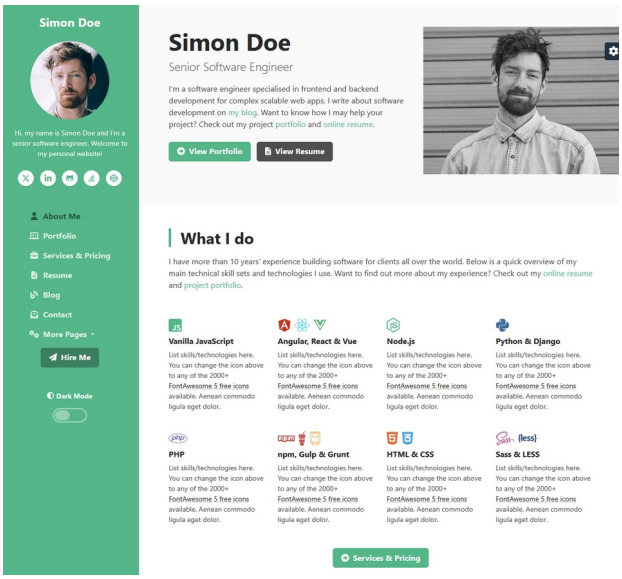
Without going into too much further detail at this time in the analysis phase, Vite+React and TailwindCSS will handle delivering said content to end-users, viewers and collaborators devices, quickly and seamlessly.

Furthermore, I monitored an array of sources that seemed to match users requirements in terms of the front-facing designs that can be implemented within our chosen tech-stack, VRTS.

We found Techhut.tv's developer blog and various other blog templates had similar requirements to our own and compiled similarities between them based on some of the user stories we gained.



(TechHut)



Features

- Minimal styling
- Mobile responsive
- 100/100 Lighthouse performance
- SEO-friendly with canonical URLs and OpenGraph data
- Sitemap support
- RSS Feed support
- Markdown & MDX support
- Syntax highlighting
- Image optimization
- Table of contents
- Dark mode
- Reading Time
- Pagefind static search library integration
- Related posts
- Share posts (Linkedin, twitter)
- Draft mode
- Copy code block

(3rd Wave Media, 2024) (Daniel, 2023)

I conducted an analysis of what our current system can handle as well as an analysis of what a future system might require. Typically, VPS packages that support small-medium websites will provide 1-2 core cpus, 1-4gb of ram for the system, up to 500gb of storage space, and generally at least 500gb upload speed from their locations (Austin, 2025). My current configuration has 5 cores, 32gb ram, two 1tb solid state drives, two 16tb drives, and is currently underutilised (James, 2024), but I can only utilize 20mbs upload and am subject to Internet Service Provider contracts and fees should the project scale locally. For a typical blog, 7400 visitors per month could be supported on 2GB of bandwidth (C., 2022), but as I only plan to host maybe 30 per month, that bandwidth scales down to only 13-20 mb per month. At any one time I may be hit with maybe 15 users, based on my class size. A general blog page after compression may be up to 112KB-300kb, so 15 x 300KB = 4.5MB (Mirchevski), so even then my 20MB upload seems to cover this requirement for the time being.

The following specifies the Functional/Non-Functional requirements consolidated by myself, the IT specialist, based off said research:

Functional Requirements		**Non-Functional Requirements**	
Blog		**Capacity**	
The website shall display an About-Me Section.	Must Have	The website shall support a peak of 50 or less concurrent sessions at launch.	MustHave
The website shall display a Blog Section with recent posts.	Must Have	**Availability**	
The website shall display Social Media Sharing buttons.	Must Have	The system should be available 24 hrs per day, 334 days a year.	MustHave
The website shall display Portfolio/Project Links.	Must Have	The system shall not lose any user content.	MustHave
The website shall be sortable by Categories/Tags.	MustHave	The system shall reflect user changes within 5 minutes of submission.	MustHave
The website must allow Embedded Code.	MustHave	The website should log users in within 5 seconds.	MustHave
The website shall have a Search Function.	MustHave		
The website should have a Comments Section per blog post.	ShouldHave		
The website should feature Responsive Design to accommodate a variety of devices. MustHave.	MustHave		
The website must feature SEO Optimization	MustHave		

These requirements seem to entail we'll have our main greeting page with the listed requirements above, as well as a blog page for each listed blog post or additional page. While similar products on the market, such as Squarespace, Wordpress, and Medium offer great scalability and performance (Main et al., 2025), they also are a bit too involved based on our

users' identified scope of simply needing to host their portfolios and collaborate respectively. This provides us a clear context for the business implications this may provide as this project aims to focus less on the content management, and more on the display of said content to end-users. Due to the scalable nature of the chosen hardware hosting and chosen tech stack, VRTS, we ensure the website will be scalable to a larger VPS system as the project's user base grows and bandwidth limitations become more apparent, detailing a need for upgrade and further testing and analysis.

If the website were to scale and elicit a buyer of the project, I could see sites like LinkedIn integrating the technology to provide a more customizable, user-friendly, platform to show off their users' work. I could see the project being sold as a hosting service for professional content creators via subscription or other means of monetization, such as advertising on the platform. While these market and business implications fall within the scope of the design of future iterations, it falls outside the scope of this project mainly due to monetary constraints, and scope of the current user base, which is currently 2 - myself and my partner. Future iterations of the project could easily deploy to a VPS system to accommodate the potential stakeholders mentioned though!

To conclude our Analysis phase, this analysis phase included gathering user stories and opinions, monitoring existing products and solutions, measuring current systems potential capacity limitations, and evaluating potential business implications to define various end-users and stakeholder requirements in building a successful development blog platform that scales past its current planned iterations. We found that the current systems will support the requirements mentioned, but also acknowledged that if stakeholder interest grows and expands, a larger scale deployment to a VPS system would be needed to handle future bandwidth limitations,

stakeholder requirements, and business implications. We addressed how each of the technologies used in our stack, VRTS, will aid in delivering a system that simply, securely, and efficiently meets users professional needs.

In our next iteration, we will experiment and collaborate on designs, wireframes, and schema to figure out further requirements and create an implementable design for our development phase to deliver the content.

Design Phase: Building a Blog with Vite + React, Strapi, and Tailwind CSS

The purpose of this document will be to define the technical architecture of our chosen tech stack, VRTS, as well as use-cases, front-end design, meeting requirements, best practices and ethical considerations, while also discussing a data dictionary of entity relationships and process logic to build an implementable plan for our development phase!

The client-side architecture consist of a front-end user interface built on top of Vite+React for its hot-module reloading and fast build times (Why vite), and TailwindCSS for its efficient scalability and responsiveness (TailwindCSS). By using a build tool on top of React, we still get all the benefits of component architecture. Each React component will represent different sections of the platform, allowing for clear naming schemes and API request moving forward, as well as ensuring future scalability and efficiency since we won’t have to rebuild components each time we’d like to reuse them. We’ll break components into sections like the navbar with links and search, the blogs landing page that will display some of the authors info and links, as well as recent or featured articles, sortable by tag and category. The authors blog page will contain components like the navbar, article card, article body with index, potential sharing and comment system, and related posts. A static ‘about me’ page was originally planned to be implemented, but was found to be a bit redundant given the other applicable info on other

pages, but will house the authors ‘about’ component and other applicable links more concisely. Pages will have to be called and loaded using Reacts Router, but will be managed by Strapi. To communicate with our back-end system, Strapi, which can utilize its own instance or a localized instance of SQLite, HTTP clients will use GraphQL API calls to fetch and display dynamic content such as blog posts and author info (Strapi, Graphql API).

The server side will utilize Strapi for providing a web-based admin GUI for management of posts, users, and other data; as well as exposing API endpoints via GraphQL for the front-end to query. Strapi also provides built-in authentification for user roles; it supports and can manage database tools like SQLite or MongoDB to manage posts, users, tags, and categories. Strapi will also allow us to custom model content types like posts, potential comment section, and portfolio pages. This allows for future scalability, as defined in our requirements analysis.

Hosting for all the various tools in our stack can be hosted via local hardware, or a larger VPS system, as they are all fully compatible with various Linux implementations. This means the hosting can be implemented on local hardware via a local node inside a private lan, using Cloudflare for proxying HTTPS requests to the server, Nginx Proxy Manager for directing HTTPS traffic in and out of the local network, and a CloudflareDDNS attached to our Nginx instance to update our infrastructures IP address to Cloudflare, given our residential ISP service.

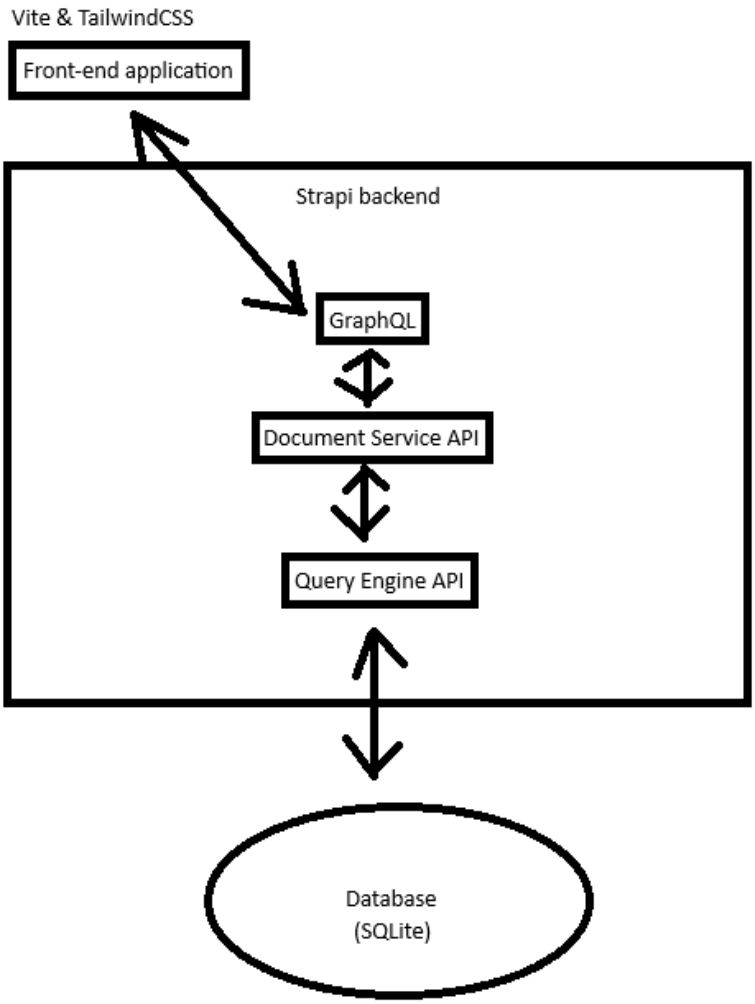
For version control, this project will be utilizing Git and GitHub, in tandem with Strapi’s content management solution, to ensure a safe and effective development environment. Continuous integration and deployment, or CI/CD, could more easily implemented in future iterations this way.

The data flow for this project has been greatly simplified in choosing Strapi for content management. The backend Strapi GUI allows content creators to add, update, delete posts,

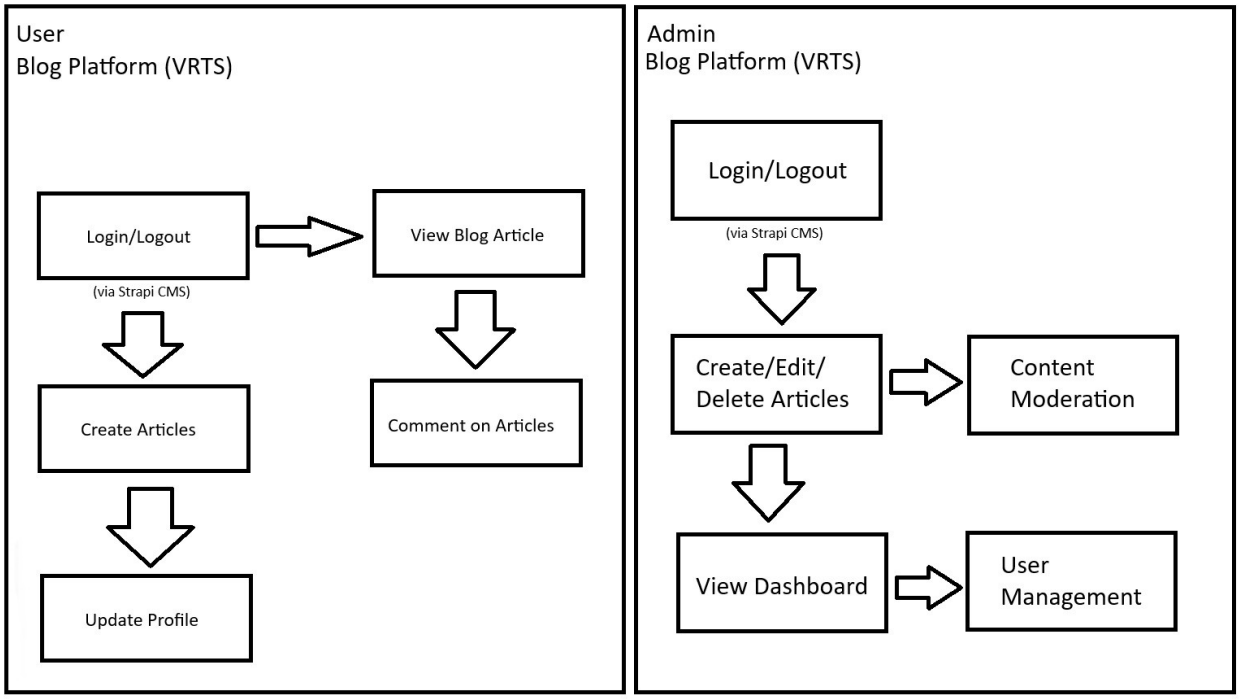
manage categories, tags, and various portfolio components, as well as content management plugins for embedding code and other nice-to-have features expected in a portfolio blog, a development blog nonetheless. As for real time updates, Strapi also contains plugins for an iteration focused on building out a comment and sharing system; but this will have to be reserved till after basic functionality is achieved with building our initial blog portfolio, as outlined in our requirements analysis.

For our security, Strapi luckily handles user permissions and roles, as well as can be set to include OAuth for secure authorization. Future research will be needed after initial implementation to build a secure front-end portal for management and further security authorization, but the back-end GUI will be reserved to local management for the time being, so Strapi's built-in management will work for us for now. HTTPS will be handed out to clients via Nginx, specifically with LetsEncrypt, for ensuring safe transmission of data over SSL in and out of the network. This meets current demands, but scales well to fit global compliance policies. Using Cloudflare for proxying traffic to our endpoint also takes advantage of rate-limiting clients and API calls, as well as various other security controls that may scale well to future iterations, as GDPR compliance and similar compliance guidelines aren't necessarily a concern for the limited current user base of two users; but again, scales well to fit them in the future!

Essentially, the front-end and back-end are independent, but communicate via GraphQL APIs. Vite+React provide the dynamic and responsive user experience, while Strapi handles the content management and database, in our case SQLite, which is the default for Strapi. TailwindCSS ensures our components scale well on different devices and are user designed and responsive.


Vite & TailwindCSS
Front-end application
Strapi backend
GraphQL
Document Service API
Query Engine API
Database
(SQLite)

(pwizla)



Currently, the users of this project will include administrators of the Strapi back-end and blog readers. Admins will use the Strapi web GUI to login/logout, moderate content, manage blog posts, manage users and accounts, as well as various dashboard analytics to track engagement! The blog viewer should be able to view posts and related links, but will be expanded in later iterations to include being to able to register//update/login/logout user accounts to deliver a managed comment system.

The main use-cases described are mainly handled by Strapi's CMS. Admins can create a new blog post by entering text, adding images, embedding code-snippets, selecting categories and tags, previewing the post, and submitting. Admins may also want to manage their authentication and authorization by visiting the Strapi back-end GUI to login, enter their credentials, the system authenticates and provides/denies access, the admin can logout at any time. The admin will also be able to manage content from the Strapi back-end where they may

login to the admin dashboard, navigate to ‘Posts Management’ or ‘Moderation’, review posts, approve, reject, edit, delete posts, and manage user activity, in future iterations. In these future iteration, users may want to login to their account, navigate to their profile page, update personal info like email or profile photo, and save changes in order to properly manage a compliant comment system.

Wireframing was conducted to address the functional requirements listed in our analysis phase. The functional requirements included:

- Shall display an ‘About-Me’ section, which will re-utilize other components of our blog when clicked.
- Shall display a a blog section with recent posts.
- Shall display social media buttons.
- Shall display links to portfolio and resume.
- Shall display Categories and Tags, in a drop-down.
- Must allow embedded code.
- Shall have a search function.
- Should feature a responsive design to accommodate a variety of devices.
- Must feature SEO Optimization.

Taking time, cost, and technical feasibility into account; Vite ensures we get a fast development environment since updates can be hot-swapped during development, React encourages re-usability and component building to reduce development time and improve accessibility in our design. These tools also ensure better maintainability since components and modules can be updated quickly. TailwindCSS allows us to use prebuilt elements directly in our JSX-React code. This reduces needed code and development time, while also encouraging

accessibility and usability standards across multiple interfaces. Strapi provides an easy to use admin dashboard, Content Management System, and customizable API to save time on developing a back-end for this project, given the scope and business implications described in the requirements analysis.

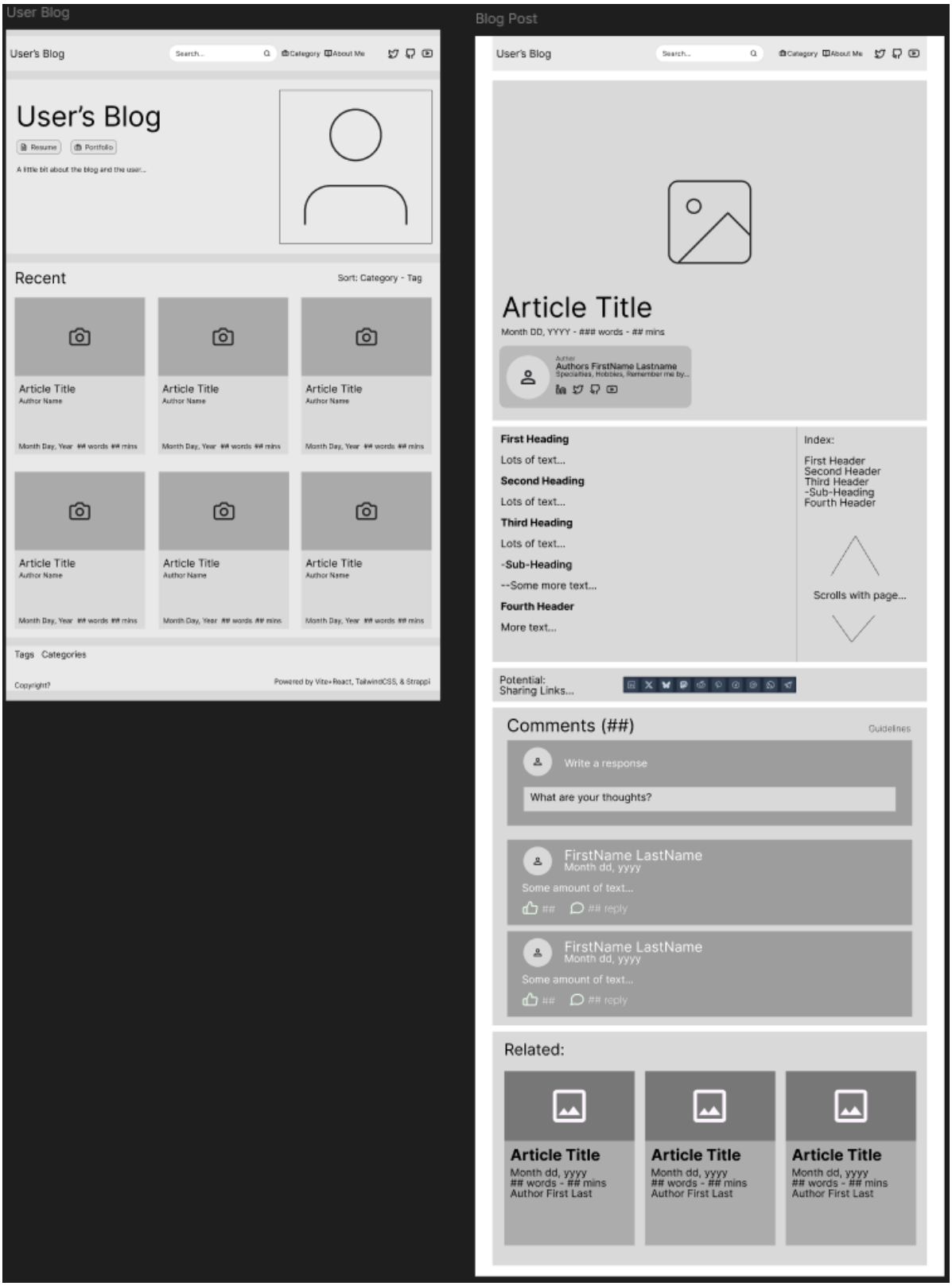
While there are licensing costs associated with using open-source software at scale and, especially, for profit, this projects current scope doesn't meet the criteria for self-hosting on my home-lab for the time being and does not currently intend to include any direct revenue streams, so our software stack is currently free to use (Strapi, strapi / license)! Furthermore, our stack is also widely and broadly supported, given the nature of open-source technologies. This means our project can still be more easily scaled to a larger VPS system, such as Hostinger or Linode, for quick, cheap, and scalable deployment should the projects scope and user base grow!

Technically, Vite is optimized for modern web standards, supports hot-module replacement, refered to as HMR, which speeds up development, debugging, and future iterations (Why vite). React, and Strapi for that matter, offer libraries and plug-ins for further state-management, routing, and data retrieval; using React Router for navigation (ReactRouter, Navigating) and Axios for managing API request (Axios API). TailwindCSS takes more of a 'mobile-first' philosophy, ensuring proper device scaling, as well as it still reduces query times and doesn't rely on custom CSS, but can still be overridden if necessary. Strapi's flexible API CMS ensures efficient management and maintanance, while also being easily integrated into our component based React app. Strapi also supports authentication, file uploads, user-roles, ensuring current security and functional standards, while allowing for future scalability!

In terms of flexiblity, by separating our front-end and back-end architectures via these GraphQL API calls, this also means that we can more easily experiment and swap out or upgrade

components of our architecture without disrupting most of the functionality of the other systems! This means if we'd like to experiment with different front-end build configurations, or we want to try another headless-CMS system, like Hugo, it should be much easier to implement should the needs arise (Strapi, Graphql API).

Given the functional requirements, the non-functional requirements listed in our requirements analysis, and all that was discussed above, the following wireframe was designed to achieve a Minimum Viable Product that meets our current needs, while also playing with ideas for future iterations, such as the comment and social sharing functions.



Compliance with Ethical Considerations in System Development

There are still a few hardware and software best practices, as well as ethical considerations to account for regarding our current and possible future scope. Some best practices include modular design, as satisfied by React and Strapi; code quality, such as using descriptive and consistent variable naming schemes and consistent organizational standards. Unit testing, which can be touched on more during our testing phase, can help test our API calls and security, such tools like Jest or React Testing Library. Using Git to keep track of our stack will also lend itself to testing and deployment.

In regards to accessibility on our blog portfolio, we can follow Web Content Accessibility Guidelines, or WCAG, during our development to encourage proper accessibility standards are being met to ensure a wide-audience has access to the platform (WAI). TailwindCSS also utilizes ‘focus’ controls and screen reading tags, such as ‘sr-only’, to help meet accessibility standards (Hover, Focus, & Other States). While more research will be performed before our testing phase, we can utilize tools like axe-core (Deque, 2024) or Lighthouse (Introduction to lighthouse, 2016) to help identify accessibility standards are being met. We can also try using various screen readers, like VoiceOver, or built-in tools to ensure accessibility.

TailwindCSS’s ‘mobile-first’ philosophy not only lends itself to accessibility standards and screen readers, but also ensures a mobile-friendly and responsive design. TailwindCSS’s text tagging controls also help adhere to accessibility, responsiveness, and SEO by sticking to standards and ensuring clearer controls.

Utilising Vite+React, we get lazy-loading and component based updates, reducing initial page loading times and improving SEO. We also ensure comparability with future optimization tools like WebP, with a tool like Image Optimization API, to reduce file sizes; GraphQL can

efficiently call what we need already by its nature, so API calls are quick and support various compression plugins. Strapi supports various content management delivery solutions for limiting file sizes and transfers.

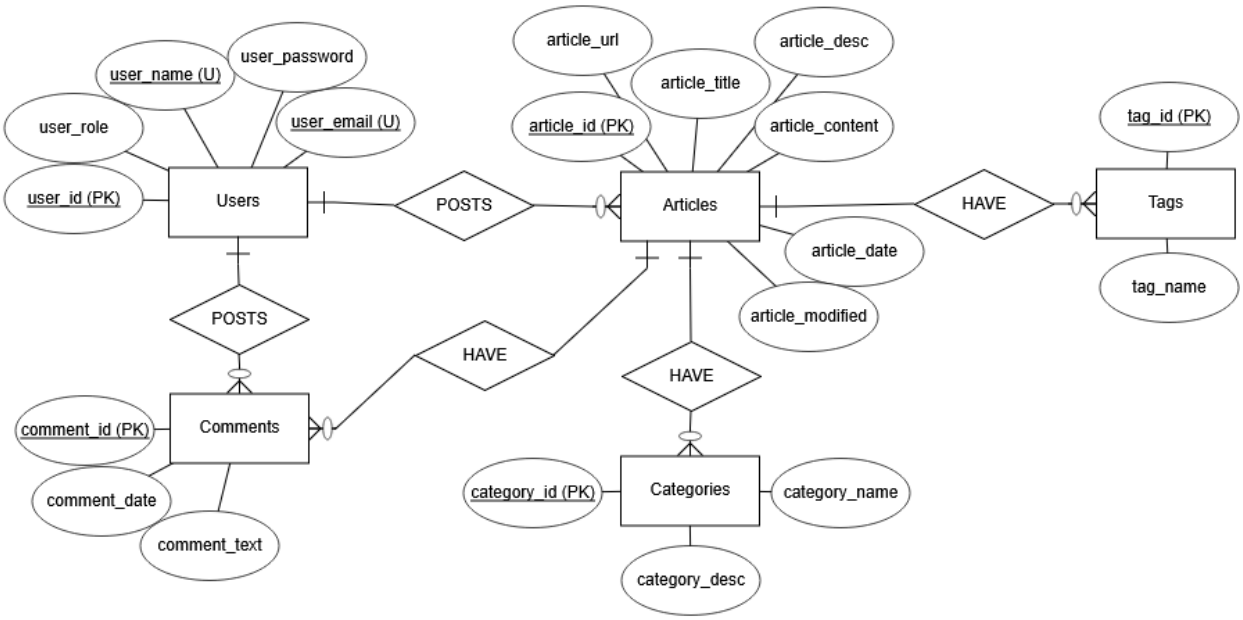
From a user data standpoint, our platform doesn't currently need to comply with GDPR regulations as the platform doesn't currently have data-collection or data-deletion concerns. By default, however, Strapi complies with said regulations by default in its content management system, so the platform should scale well to follow such standards (Strapi.io). Encryption of admin authorization and like controls are handled by Strapi! Our communications will be handled and proxied over Cloudflare and funneled to our Nginx Proxy Manager with LetsEncrypt for HTTPS to aid in the current scopes admin privacy and encourage scalability and safe behavior and development. Further iterations will be needed for a more robust security audit given the current scope of the project.

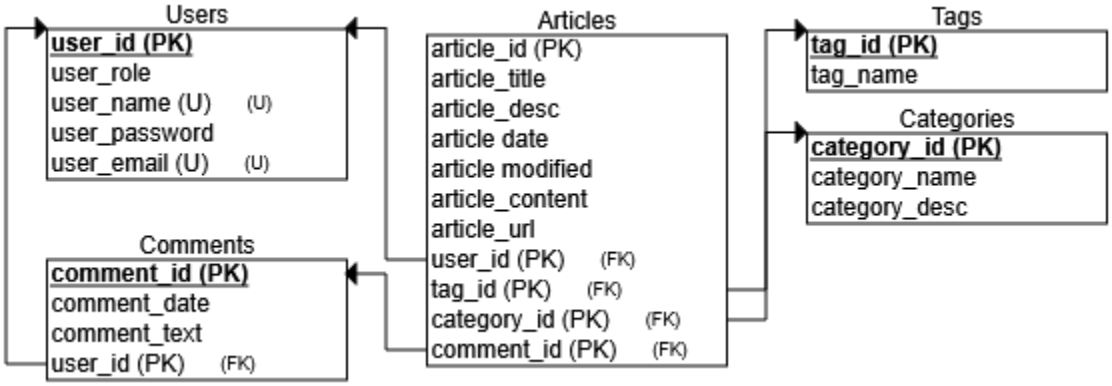
With our use of open-source tools we get all the benefits of community, security, and scalability, but these tools do ask for attribution, and as discussed earlier in the document, may include licensing at larger scales not attained by this projects scope yet. However, with open-source software, by utilizing and leveraging these open-source tools for my project, I lead with the philosophy of community and documentation that seeks to give back to said community, which may lend itself to a larger role in its development, such as feature request and contributions in the future.

In conclusion, we defined the technical architecture of our chosen tech stack, VRTS, as well as use-cases by users, admins, and viewers; as well as the front-end design via wireframes, meeting requirements, best practices and ethical considerations, while also providing an Entity Relationship Diagram, process logic, and a data dictionary to build an implementable plan for

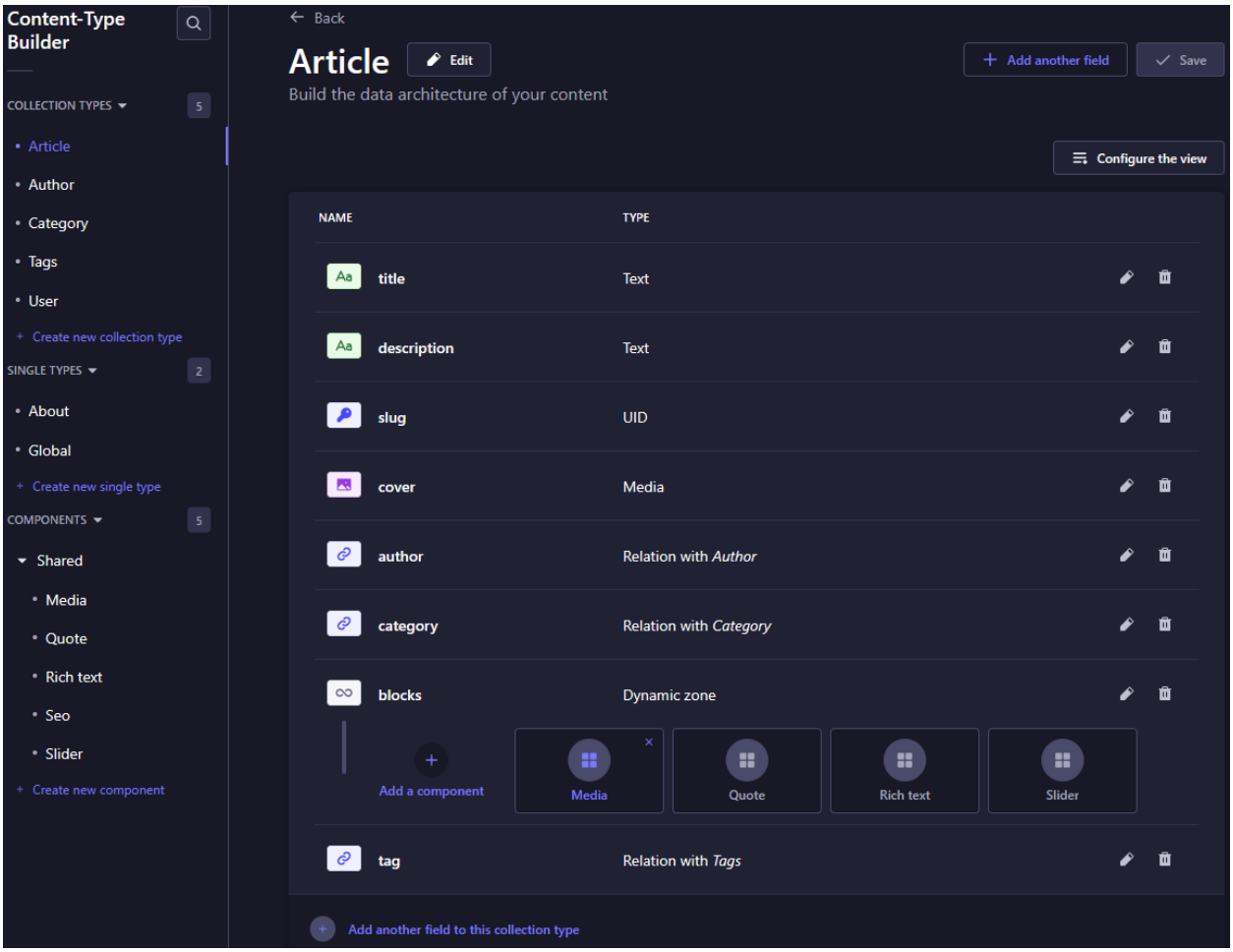
our development phase!

Entity Relationship Diagram & Relational Schema of Strapi managed database - so-far!:





Installed Strapi CMS Dashboard:



Development Phase: Building a Blog with Vite + React, Strapi, and Tailwind CSS

In this phase we will document and build a full-stack web application using JSX with Vite+React to develop and manage our projects front-end, TailwindCSS for in-line styling, Strapi and GraphQL for back-end integration, now with the help of the Apollo client for GraphQL API calls, ensuring an efficient and maintainable project! The purpose of this document is to document the installation of all the hardware and software development tools tools used in our development environment; the functional features, programs, and databases involved; building the necessary API interfaces; as well as a discussion of how our technical architecture and code development support high-quality technical performance measures in relation to usability, scalability, accessibility, availability, and reliability.

We initially set-up an Ubuntu 22.04 container on our local Proxmox instance called ‘dev-blog’. The machine, so far, has been given 32GB for the boot-disk across 2 NVME drives mirrored in ZFS for redundancy and speed; 500GB for the local data storage mounted as ‘/devblog’ on the root, also mirrored in ZFS for data redundancy and reliability, comparable to many free-tier VPS offerings; as well as 4GB of ram to start and 2 of our 5 cpu cores from our 12th gen Intel 8505 Pentium Gold processor.

We then ran a few commands to update and upgrade our container using the ‘apt install update && apt install upgrade -y’ to get all the latest packages and security updates. We added our local user using the ‘adduser’ command, filled in the requested info, and then added the user to the ‘sudo’ group using the same ‘adduser’ command after user-creation. We then switched over to said user profile using the ‘su’ command with our username in order to limit root access and prompt for our users password each time we make important changes to the server. Our user must now use ‘sudo’ commands in order to make privileged changes to our server. This means the user only has write-access over their own home directory and the /devblog directory, by using the ‘sudo chown USERNAME /devblog’ command. This largely protects the main operating system components running on our boot-disk and gives us better log details attached to our specific user should anything occur to our project.

We then installed Fast Node Manager, or fnm, to set-up and install Node.js and our Node package Manager, or npm, using the following commands:

```
# Download and install fnm:
curl -o- https://fnm.vercel.app/install | bash
# Download and install Node.js:
fnm install 22
```

and we verified our installation using:

Verify the Node.js version:

node -v # Should print "v22.14.0".

Verify npm version:

npm -v # Should print "10.9.2".

To create a new Vite project, we ran ‘npx create-vite devblog’ in our root /blog directory, in which we chose to create a React JSX project. To install and keep track of React and TailwindCSS depandancies we moved into our new ‘/blog/devblog’ directory and ran the following npm commands to ensure updated dependancies:

npm install react react-dom

npm install tailwindcss

To install Strapi globally and create a Strapi project inside of /blog/devblog/devblog-cms we ran:

npm install -g strapi

npx create-strapi devblog-cms

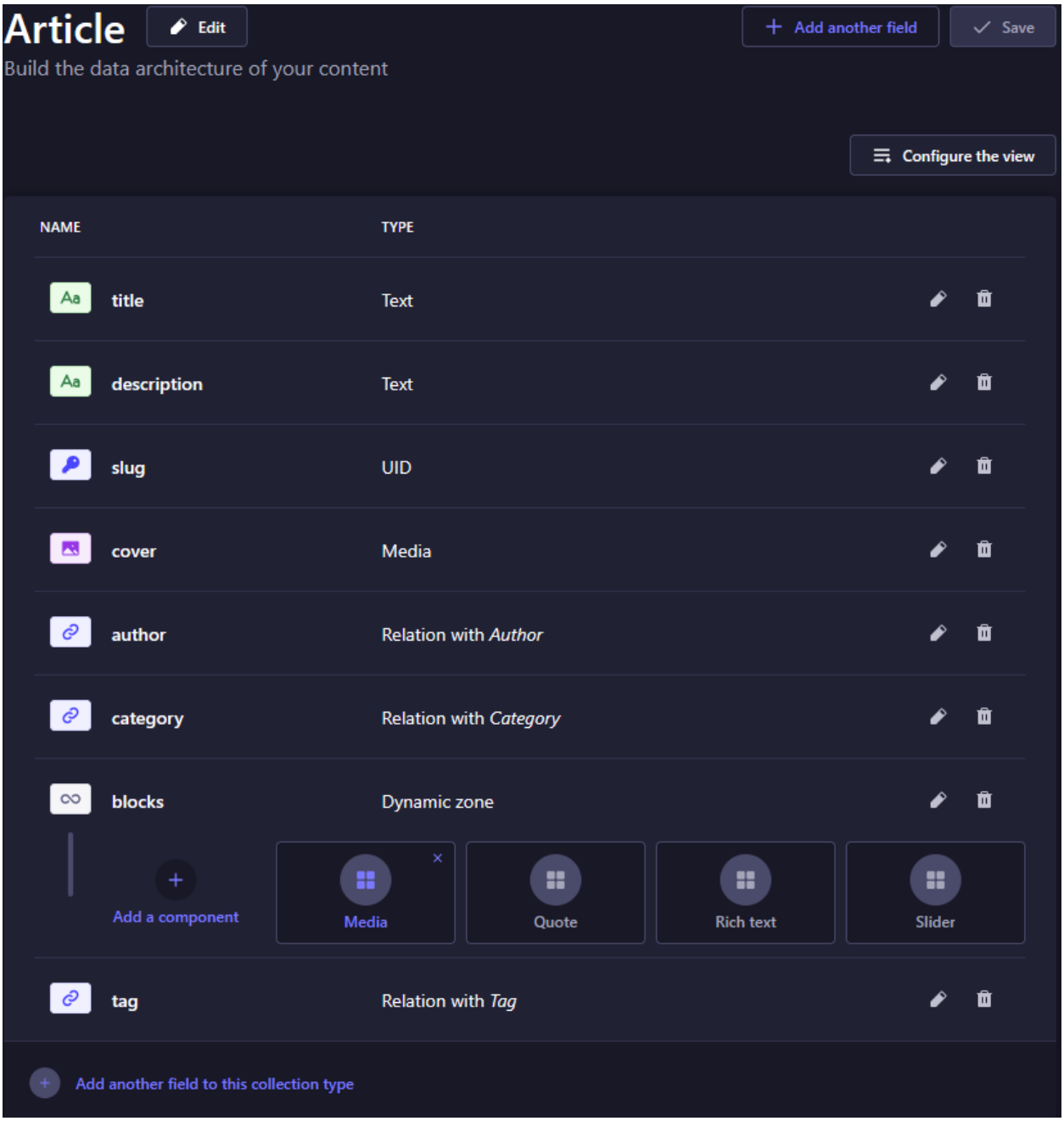
Via the command line, Strapi walked us through the set-up process. Since MongoDB is not supported and not necessary for our newly developed project, we chose to stick with Strapi’s default database, sqlite, as the flavor of SQL doesn’t particularly effect our needs in this case. We opted to start the project with example structure and data, opted out of Typescript support, since I am not familiar, and opted to install dependencies using npm.

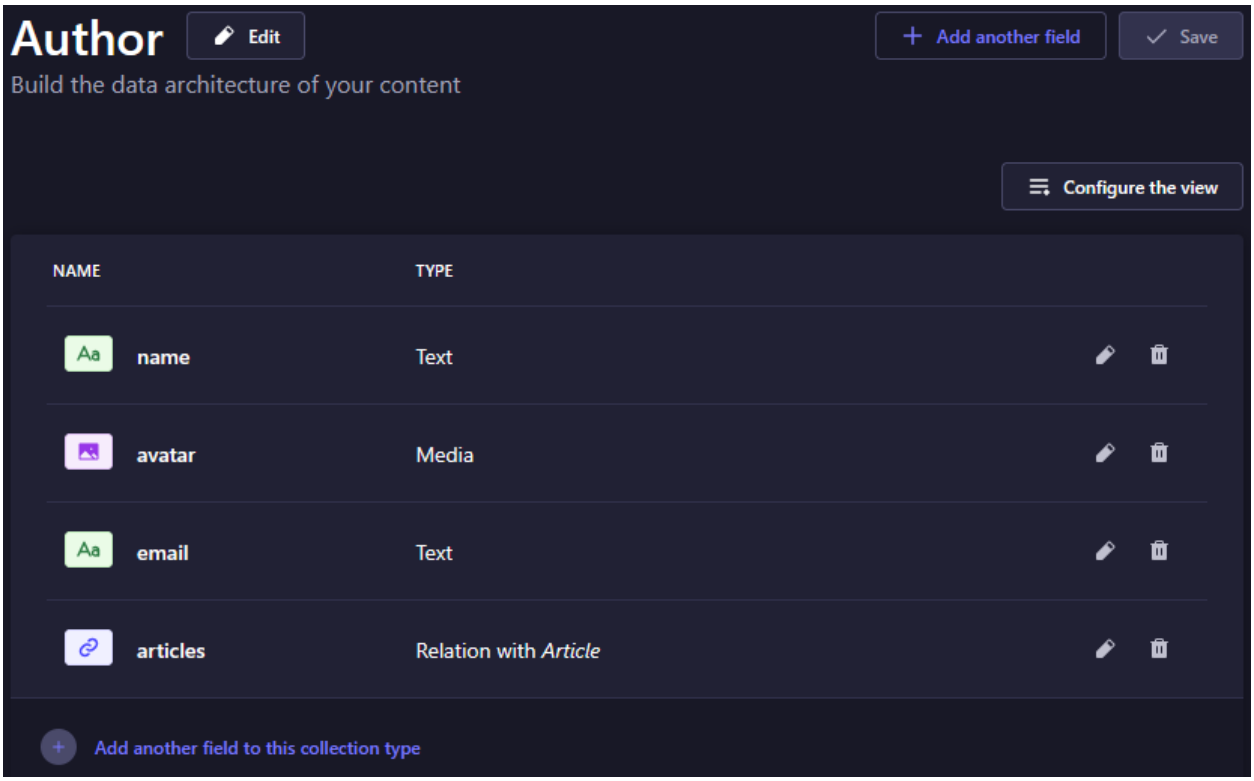
```
? Please log in or sign up. Skip
? Do you want to use the default database (sqlite) ? Yes
? Start with an example structure & data? Yes
? Start with Typescript? Yes
? Install dependencies with npm? Yes
? Initialize a git repository? Yes
```

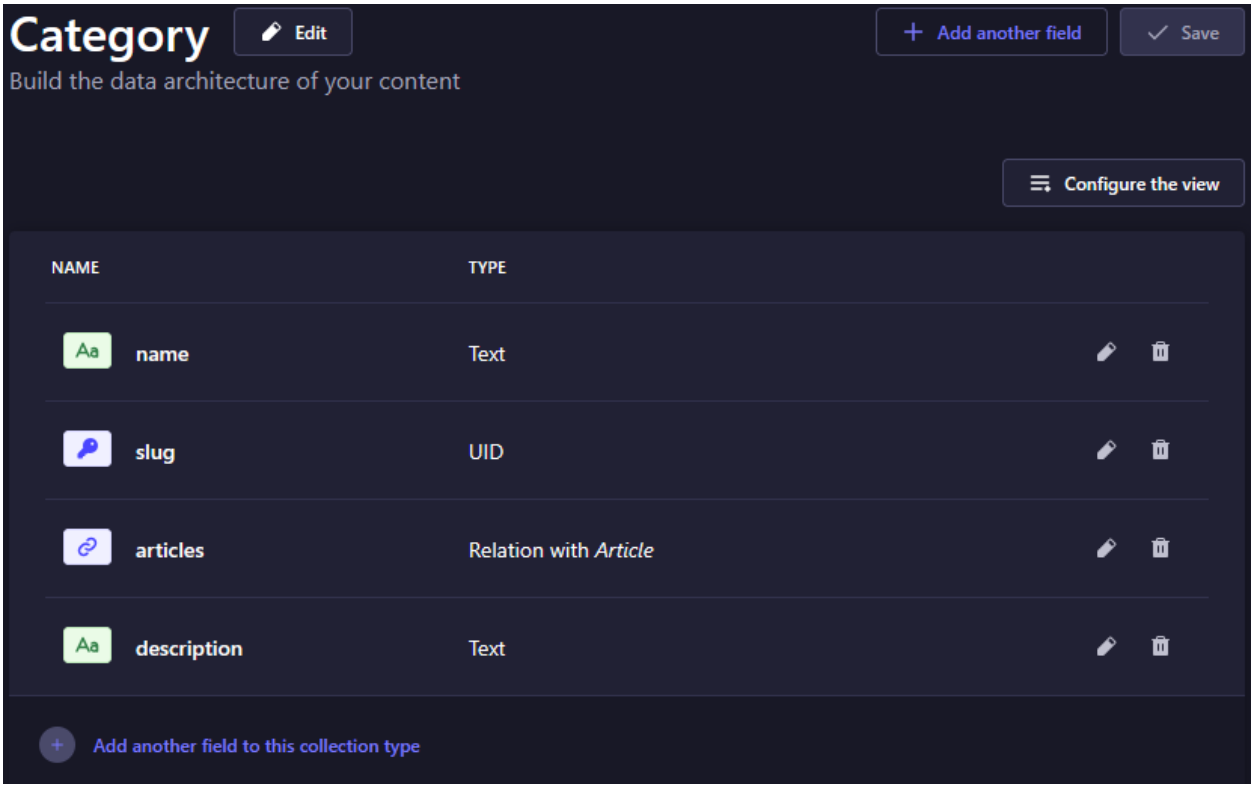
(Strapi)

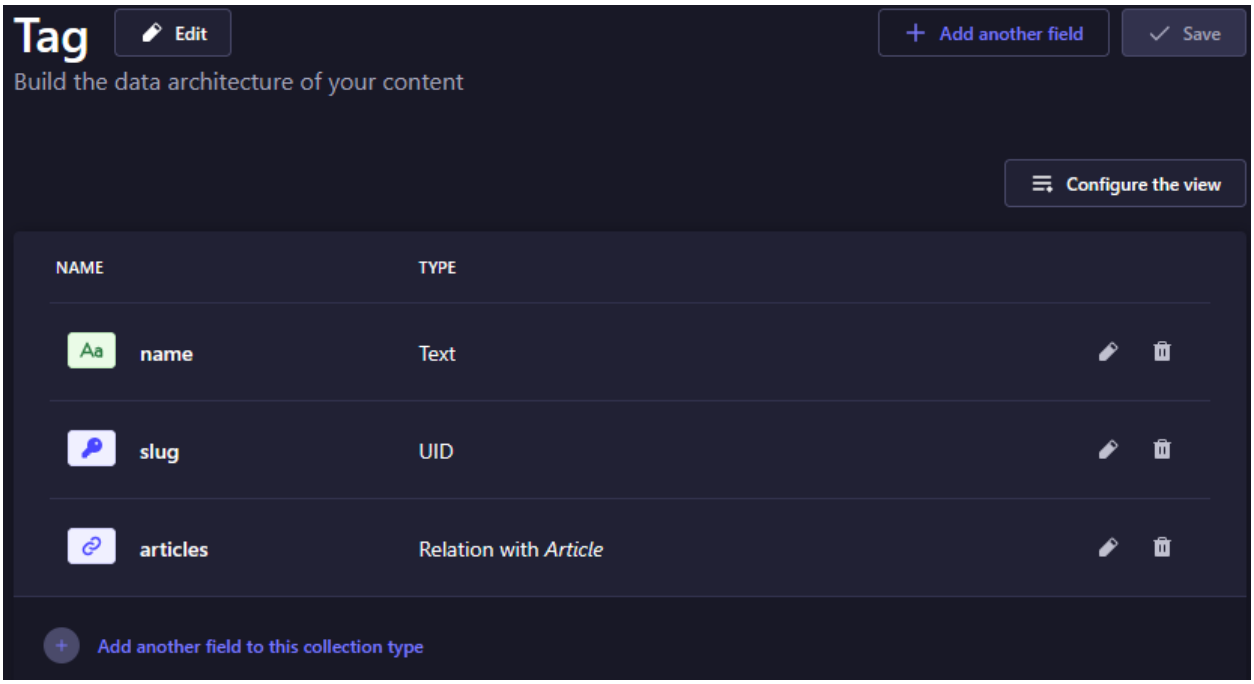
To install and enable support for GraphQL API integration we ran ‘npm install @strapi/plugin-graphql’. This plugin can be accessed, once the Strapi server is running, at ‘localhost:1337/graphql’. The Strapi GraphQL plugin is an embedded Apollo Server playground, so we will install and use the Apollo Client, respectively, to make GraphQL request in our app.

To manage our back-end content, we then started our Strapi server by running ‘npm run develop’ inside of our /blog/devblog/devblog-cms directory. This opens a local server at port 1337 that we can navigate to in order to manage our back-end and add blog posts. Based on our ‘Entity Relationship Diagram & Relational Schema’ from our Design phase, we reviewed and tweaked the default Strapi content-types to better match our current local scope, as seen below:

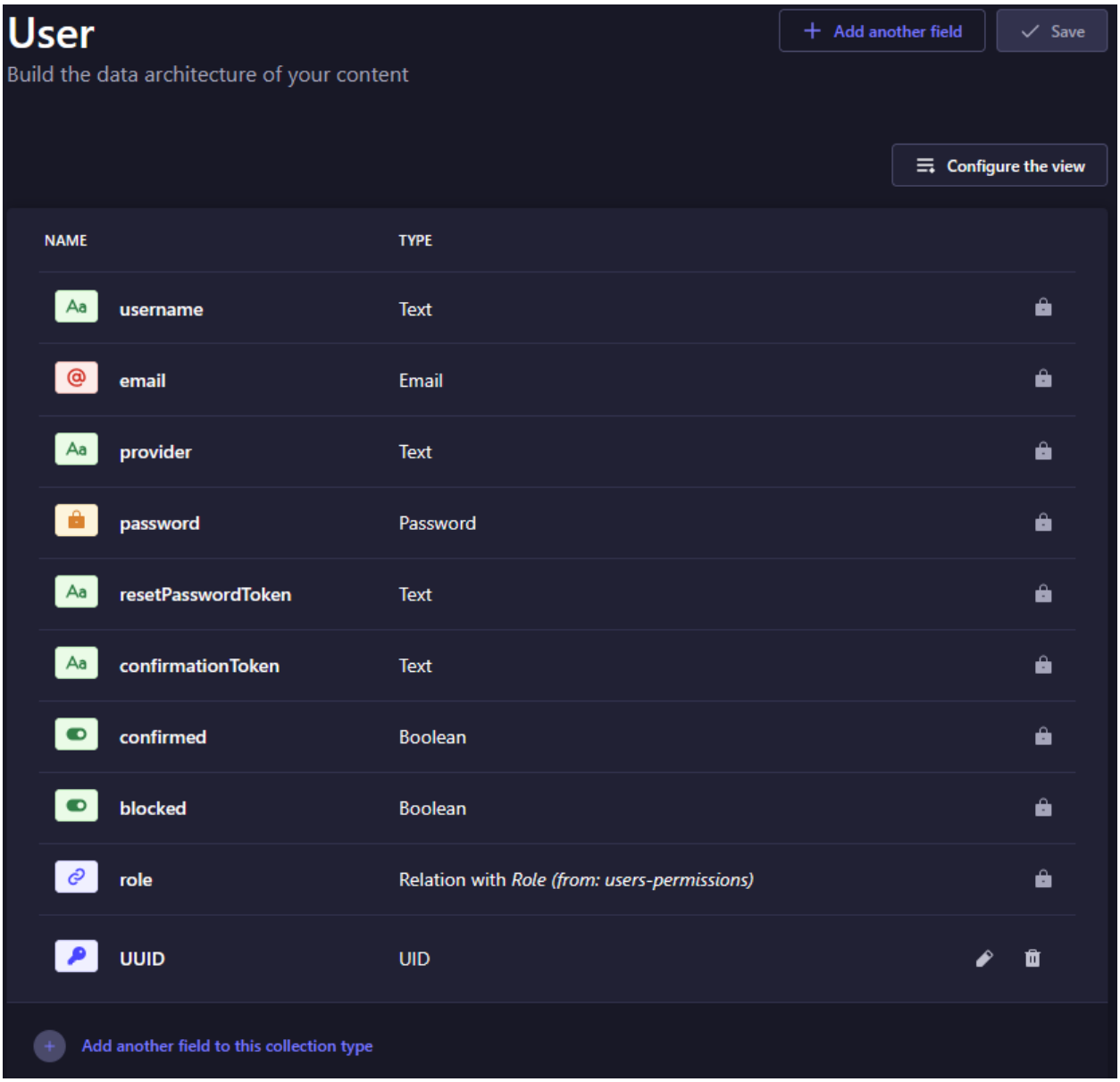

Article
Edit
Add another field
Save
Build the data architecture of your content
Configure the view
NAME
TYPE
title
Text
description
Text
slug
UID
cover
Media
author
Relation with Author
category
Relation with Category
blocks
Dynamic zone
Add a component
Media
Quote
Rich text
Slider
tag
Relation with Tag
Add another field to this collection type


Author
Edit
Add another field
Save
Build the data architecture of your content
Configure the view
NAME
TYPE
name
Text
avatar
Media
email
Text
articles
Relation with Article
Add another field to this collection type

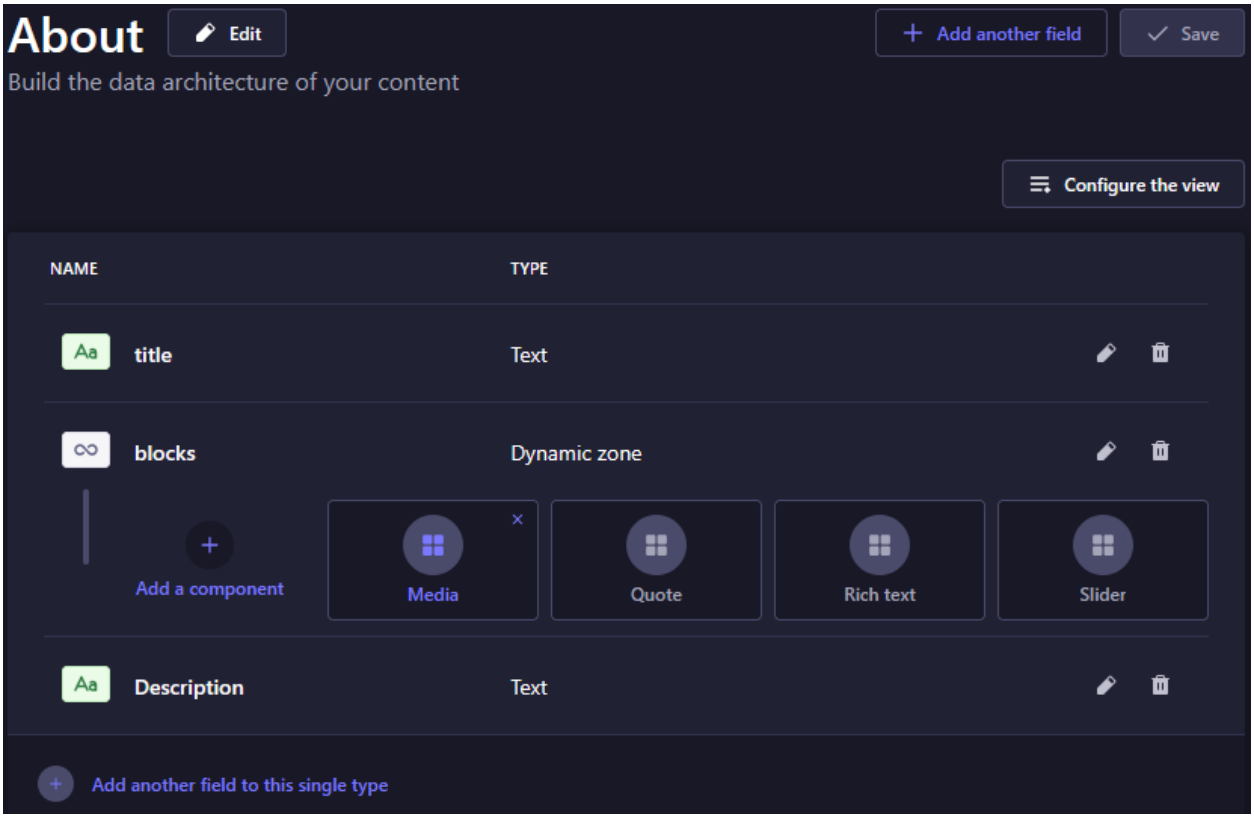

Category
Edit
Add another field
Save
Build the data architecture of your content
Configure the view
NAME
TYPE
name
Text
slug
UID
articles
Relation with Article
description
Text
Add another field to this collection type



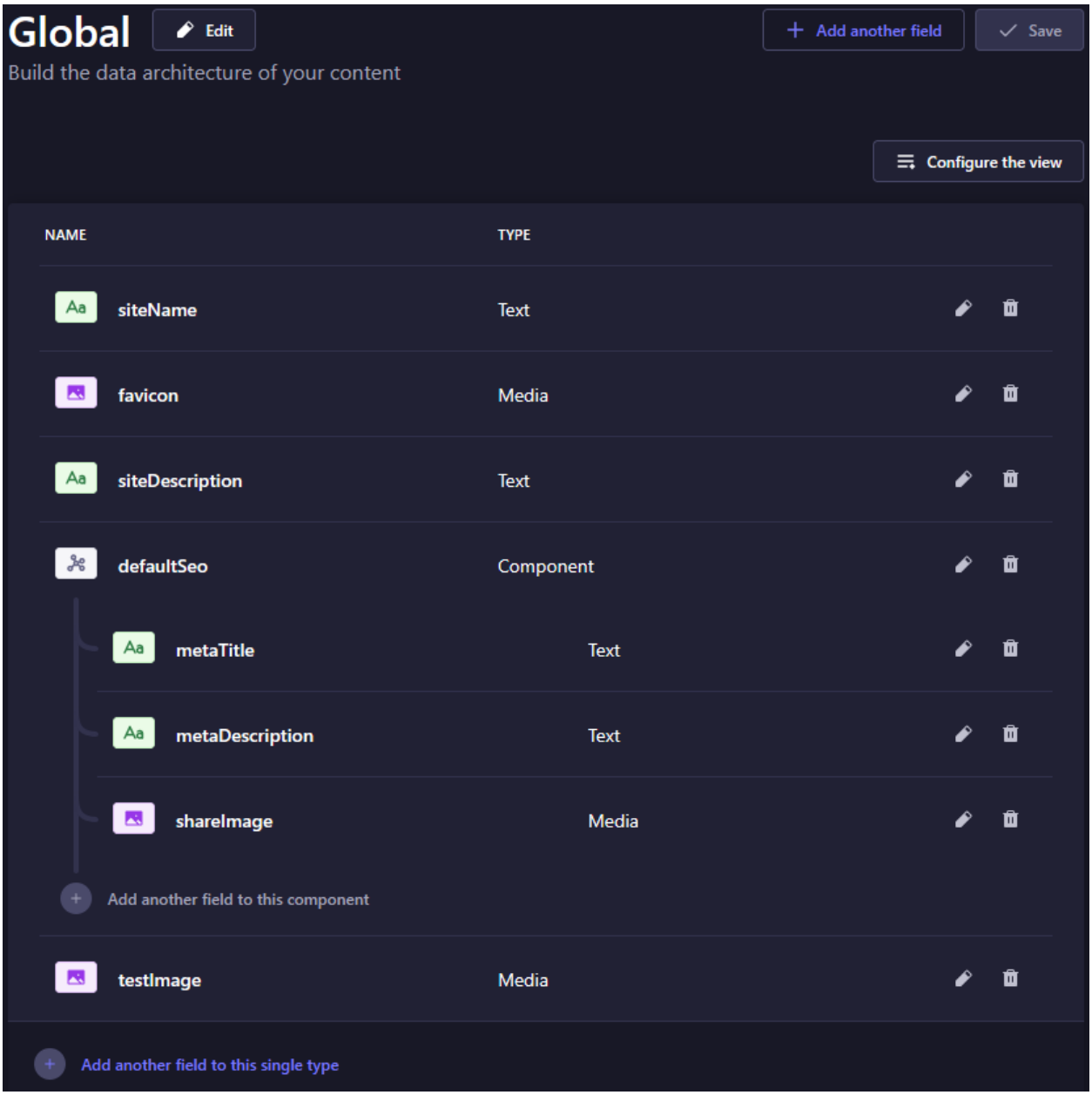
We've opted to use most of Strapi's default content-types and values since they meet the needs of our current scope; but currently unnecessary components, much of which found in our 'User' content-type, scale well to fit future iterations in scope. The Category tag was edited to properly relate to our Article type. The Tag type was added, as per our design specficications, but was found to be redundant at the projects current state, given the limited amount of articles served and the specificity the Category tag already gives us in organizing our content. Tags make better sense when one wants to add further specificity to their document that more common and existing categories may not necessarily fit. However, we added the Tag type anyways to scale better to future iterations where introducing Tags would make better sense.



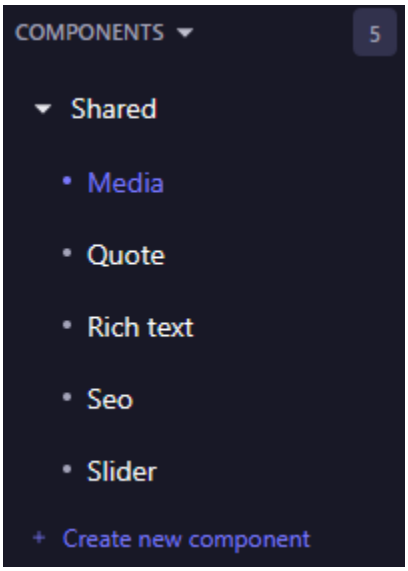
The User type will also not be used in our current iterations, given the limited number of admin users. Admin users login using the Strapi Admin Dashboard and are configured and managed by our Strapi server. However, in a larger deployment with an active user-base, the project will likely want to open up to allow other users to sign-up and manage and serve their own content. To satisfy future requirements and to increase our understanding for future iterative work, we've opted to keep Strapi's default User set-up type, which we can easily relate to Author later on as well.



Our design phase also found the About page to also be a bit redundant given the nature and scope of the current project, but has been added for its simplicity to experiment with routing using the React Router DOM plugin from React.



We have also opted to keep the Global type as it gives us a space to put assets that build the brand of the website and provides some basic search engine optimizations, or SEO, to help us get started in our current iterations, but give us a leap point from which we can continue to build and optimize our SEO, as per our requirements.



We've opted to keep the default components as they also allow us to dynamically adjust and add content to our posts, as well as provide the necessary markdown and components we may want to include at any given time. This fits our current needs to display creative content and provides us the tools to format referenced code to our liking, as per our requirements. Further research will be needed to add better code formatting and highlighting, but can be more readily applied and scaled given the component control I our Strapi Admin panel.

While our Strapi server is running, we can visit 'localhost:1337/graphql' to ensure our Apollo GraphQL server is running and is being properly managed by our Strapi server. The GraphQL Apollo playground lets us test our GraphQL queries, but I wanted to learn how to use Postman for testing our GraphQL calls. Postman is used to design, build, test, and collaborate on APIs (Postman, 2025). It provides a clean interface to test that our GraphQL endpoints are visible and query-able to the public; and it lets us ensure that restricted queries are being restricted by our Apollo server.

At first navigating to localhost:1337/graphql in Postman yielded little result. We had to go back into the Strapi Admin panel and allow public read access to our API endpoints by

navigating to Settings > Roles > Public > Edit and we had to click each content type to allow ‘find’ and ‘findOne’ API access (Postman, 2025), as exampled below:

After which we were able to find and query our GraphQL Apollo endpoint to get back various bits and pieces of our test content in Postman, validating our GraphQL endpoint works and Strapi back-end set-up will be able to serve content to our front-end application through GraphQL!

```
query Articles {
    articles {
        documentId
        title
        description
        slug
        blocks {
            ... on ComponentSharedMedia {
                id
            }
            ... on ComponentSharedQuote {
                id
                title
                body
            }
            ... on ComponentSharedRichText {
                id
                body
            }
            ... on ComponentSharedSlider {
                id
            }
        }
        createdAt
        updatedAt
        publishedAt
        author {
            documentId
            name
            email
            createdAt
            updatedAt
            publishedAt
        }
    }
}
```


```
{
    "data": {
        "articles": [
            {
                "documentId": "hzr5zmg9sz5spplh5u7veyi9",
                "title": "A bug is becoming a meme on the internet",
                "description": "How a bug on MySQL is becoming a meme on the internet",
                "slug": "a-bug-is-becoming-a-meme-on-the-internet",
                "blocks": [
                    {
                        "id": "12",
                        "body": "## Probant \n\nse Lorem markdownum negat. Argo *saxa* videnda cornua
                    },
                    {
                        "id": "7",
                        "title": "Thelonius Monk",
                        "body": "You've got to dig it to dig it, you dig?"
                    },
                    {
                        "id": "7"
                    },
                    {
                        "id": "13",
                        "body": "## Spatiantia astra \n\nFoeda, medio silva *errandum*: onus formam m
                    },
                    {
                        "id": "6"
                    }
                ],
                "createdAt": "2025-03-05T04:45:57.337Z",
                "updatedAt": "2025-03-05T04:45:57.337Z",
                "publishedAt": "2025-03-06T20:18:09.777Z",
                "author": {
                    "documentId": "xi6lu72ddfshh8p70tcrum1i",
                    "name": "Sarah Baker",
                    "email": "sarahbaker@strapi.io",
                    "createdAt": "2025-03-05T04:45:53.491Z",
                    "updatedAt": "2025-03-05T04:45:53.491Z",
                    "publishedAt": "2025-03-05T04:45:53.489Z"
                }
            },
```


Hooray! This not only proves our process will work on the back-end, but also provides us meaningful reference for when we add our queries in the front-end using our Apollo client!

To keep track of our project and protect the work we’ve done so far in our project

directory, we installed Git by running ‘sudo apt-get install git-all’ and verified the install using the ‘git version’ command. Running:

git config --global user.name "username"

git config --global user.email "email@email.com"

These commands hold the same credentials used on Github for easy reference and is global on this machine since I am the sole owner.

We then navigated back to our /blog/devblog folder which houses our Strapi project in /blog/devblog/devblog-cms. By initializing Git in this folder, by running ‘git init, we get Git control over our entire project, providing safe points of re-entry should a problem occur in our project. Our self contained file structure and services compliment Git’s, giving us safer control over our projects development.

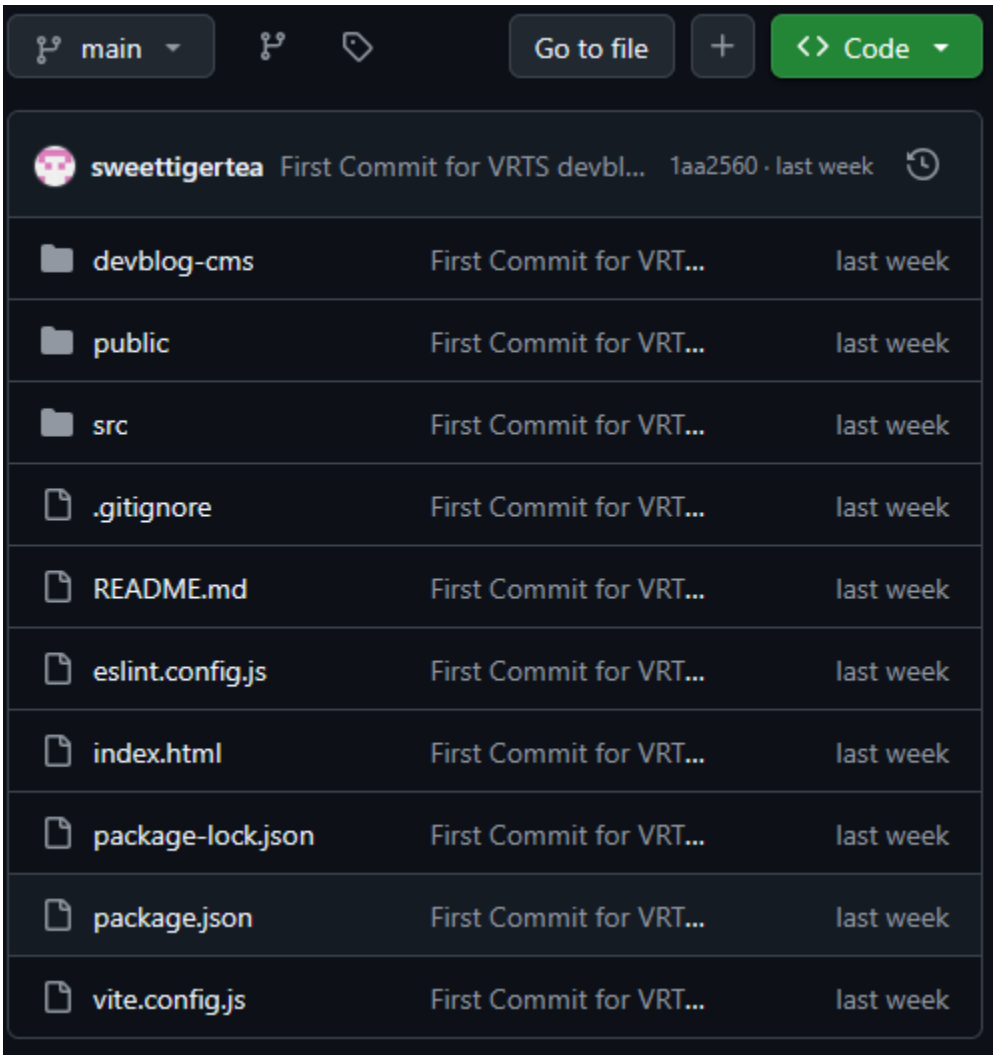
Then we ran ‘git add –all’, to add the entire /blog/devblog directory to our staging environment in Git. We ran ‘git commit -m "First Commit for VRTS devblog!"’ to commit to our first Git repository!

```
commit 1aa2560e8aff629bce4da265838dc09342941959 (HEAD -> development, origin/main, main)
Author: sweettigertea <sweettigertea@gmail.com>
Date:   Thu Mar 20 22:27:20 2025 +0000

    First Commit for VRTS devblog!
```

We then branched off into a new branch called ‘development’ using the ‘git branch’ and ‘git checkout’ commands, the results of which are seen above, in order to protect the main branch from any changes we make during development until we are specifically ready to merge them.

Then we created private online Github repository to push our Git repository to, to further protect our work and to later make public once the current project is nearer completion.



To push our Git repository to our Github repository, we had to create a personalized token on Github in order to give us access to our account when we login to our account in the command line (Daemes & STA, 2024). We went our User settings > Developer settings > Fine-Grained Token in which we created a 30 day token that allows all the necessary permissions needed to view and edit our repository; including read/write access to metadata and read/write access to our code, commits, merges, and pull request. All other permissions, including user permission for the account, are disabled for secure management of our project. The generated token is used in place of our password when we make our first push command to the origin, located at the specified Github URL below, and is saved and encrypted locally, if needed, and

can be renewed.

By running ‘git remote add origin https://github.com/sweettigertea/VRTS-dev-blog.git’ we were able to create an origin branch at the located url. To push our Git repository to the origin URL, we ran ‘git push --set-upstream origin master’ and were prompted for our username and the token we generated earlier. This allowed us to achieve the pictured result above on Github.

All that’s left is to develop the front-end. To do this, we wanted to connect to our server using our preferred integrated development environment, or IDE, VSCode. There are many ways to do this, but SSH proved to be the most flexible and reliable way integrating VSCode into our workflow. While some complications arose, since we are new to server development, we ended up installing OpenSSH server by running ‘sudo apt install openssh-server’ in the command line; then we ran ‘sudo ufw allow ssh’ to allow it through the container’s firewall (Linuxize, 2020).

Our Windows machine was also not configured to run the OpenSSH client, so we went to Windows Settings > Apps > Optional Features > Add a feature and then we selected ‘OpenSSH Client’ and installed (Reinders, 2021). This took a while for whatever reason, but allowed our system to utilize SSH services.

After this complication, we started up VSCode and went to Extensions and downloaded the Remote Development extension pack, which includes the Remote – SSH plugin, as per VSCodes recommendation (Microsoft, 2025). Finally, we were able to connect to our server remotely in VSCode by navigating to the bottom left where it says ‘Remote’ then ‘Connect to Host’ in which we were able to login using SSH! This allows us to login to our local user account locally in VSCode, and more importantly, allows us to have multiple terminals for running the Vite and Strapi servers separately during our development, as well as access to all our plugins and the benefits that come form using a proper IDE. We get to use tools like Prettier

to help format our code consistently and other plugins that help us write clean code for our specified technologies! This development style should help us learn and commit code quicker, lending itself to the projects steady development, even in the face of set-backs in our understanding.

For the front-end, more research was needed in order to deepen our understanding of JSX syntax, especially when switching between HTML/XML, TailwindCSS, and Javascript in the same document, but has been largely remediated by following along with a few videos working in JSX (Web Dev Simplified, 2023). Notably, we re-learned the importance of exporting and importing functions and components in order to utilize them elsewhere in our code…

We also ran into problems using BrowserRouter as ‘Router, Route, Switch’, but further research revealed changes have occurred and the import of BrowserRouter from the react-router-dom now uses BrowserRouter as ‘Router, Route, Routes’ to move between pages without triggering a reload of the page. Using BrowserRouter allows us to navigate between pages of the application without even having to refresh the browser! This allows quick and nimble navigation of our application without slowing down the systems with necessary refreshes and calls to the server, as well as it further encourages a component-based development strategy (Web Dev Simplified, 2022).

While we initially ran into problems understanding some of the React hooks useEffect and useState, remediated by Web Dev Simplified; our application actually uses GraphQL, which means we must import and use Apollo’s hooks, like useQuery and ‘gql’ from an installed Apollo client in our development environment. We installed the Apollo client package and the GraphQL package by running ‘npm install @apollo/client graphql’ in our /blog/devblog directory. This allows us to use the ‘import {ApolloClient, InMemoryCache, ApolloProvider} from

“@apollo/client”’ command in our applications front-end in order to apply simple, secure, and specific GraphQL queries in our front-end! We can also wrap our BrowserRouter’s Router with the ApolloProvider tag to provide our GraphQL URL to our entire project and efficiently caches responses in memory (Net Ninja, 2021)! This methodology also follows our goal to eventually deliver an efficient and performant application!

I also had some issues running all my development services at once while making larger changes to the code-base during my research and personal testing. Whether it be an issue of running this server in an LXC container, more research will be needed; but Vite seemingly doesn’t like large changes to it’s modules during deployment as it seems to have some issues hot-swapping the effected modules all at once in conjunction with our other tracked services. This means we should make more light-changes to our code-base as we go. This philosophy is complimented by Git’s version control system, so it stands that our development pattern should follow.

Finally - so far, I also had a bit of confusion about how to implement and organize my project so that I am properly encouraged to break my application into as many usable components as possible. I questioned how I should structure and name my directories so that I might be able to maintain and scale the project if wanted. This led me to research and decide a project folder structure for myself, including breaking my wireframes down into smaller usable components, the results of which you can view below:

Folder Structure:

- /src
  - /assets
  - /components

        - /tests
        - /form
            - /tests
        - /ui
            - /tests
    - /pages
        - /tests
    - /hooks
        - /tests
    - /utils – for pure, or simple functions
        - /tests

    Component/React Structure – so far

- Homepage – User-Blog
    - Header
        - Navbar
            - User-Blog-Title
            - SearchBar
            - Category-Link
            - About Me
                - UserBlog-Card-Extended?
            - Social-Links
                - BlueSky-LinkIcon
                - Github-LinkIcon

        - Youtube-LinkIcon
    - UserBlog-Card
        - Resume-Button – link to Resume-Page
        - Portfolio-Button – link to Github (currently)
        - About-Blog
        - User-Photo
    - Recent-Articles
        - Sort:
            - Category
        - Recent-Article-List – up to 3x4 grid, or 12 articles
            - Article-Card
                - Article-Cover
                - Article-Title
                - Author-Name
                - Article-PostDate
                - Article-WordCount
                - Article-ReadTime
    - Footer
        - Categories-Link
        - Fake Copyright?
        - Powered by VRTS
- Article-Page
    - Header

        - Navbar
    - Article-Card
    - Index
    - Body
    - Sharing-Links
    - CommentsSection?
        - Comment(##)
        - Guidlines-Link
        - UserResponse-Card
            - UserPhoto
            - TextPrompt – “Write a Response”
            - TextInput – “What Are your thoughts”
        - Comment-Card
            - User-Photo
            - User-FullName
            - Comment-Date
            - Comment
            - LikeComment-Button
            - ReplyComment-Button
    - Related-Articles – 1x3 (plus scroll-able up to 6?)
        - Article-Card
    - Footer

Analyzing some these pieces of our website structure can lead us to break our reused

components further, so that they can be separated and imported as needed in our application.

Repeated or Encapsulated Components:

- UserBlog-Card
- Article-Card
- Comments-Card
- RecentArticles-Grid
- Navbar

Other components can be structured within their respective folders based on the page they belong to. Subsequently, the a /tests folder will be added to each sub-folder as needed when doing our testing to ensure an understandable and scalable project. Breaking our project down structurally also lends itself as a better reference when building our front-end application and helps sectionalize our pages for screen-readers, encouraging accessible development!

While both life and some of the set-backs discussed have set us a bit behind where we initially planned to be in our project at this time, taking extra time to further research and think about the functional tools, structure, and components of our project helps us both to catch up in our development phase and greatly lends itself to ensuring I am validating my code as I continue to work. This encourages the code coverage I hope to deploy and discuss further upon the completion of our testing phase!

To conclude our design phase, we've set-up, deployed, and tested our back-end systems and methodologies; we've discussed some of the challenges we've faced in our learning journey so far; we've discussed some of the lessons we've learned and revisited our design; and we've discussed how our current and iterative development contributes in relation to usability, scalability, accessibility, availability, reliability throughout this document to exemplify and

encourage ethical development and learning that scales well the bigger the project becomes.

Testing Phase: Building a Blog with Vite + React, Strapi, and Tailwind CSS

Due to a few setbacks in our knowledge and understanding of our stack, discussed further in our Development phase, the beginning of the testing phase truly marked the beginning of our front-end development, and circumstantially provided many more opportunities for learning and testing!

To keep this document brief, we will skip a lot of the minute-technical challenges we faced during development, as a lot can be gleaned from our now public GitHub! We will instead discuss the methodologies used to conduct testing during the development of our front-end and the challenges faced in order to validate and further research the functionality of our blog platform against our requirements; specifically, ensuring the platforms robustness, usability, and reliability across various devices and user interactions throughout our application.

To conduct our testing, we spent some time analyzing our business requirements, as well as conducting end-to-end testing, integration testing, and user-acceptance testing.

As documented in our analysis phase, the requirements were as follows:

Functional Requirements		**Non-Functional Requirements**	
Blog		**Capacity**	
The website shall display an About-Me Section.	Must Have	The website shall support a peak of 50 or less concurrent sessions at launch.	MustHave
The website shall display a Blog Section with recent posts.	Must Have	**Availability**	
The website shall display Social Media Sharing buttons.	Must Have	The system should be available 24 hrs per day, 334 days a year.	MustHave
The website shall display Portfolio/Project Links.	Must Have	The system shall not lose any user content.	MustHave
The website shall be sortable by Categories/Tags.	MustHave	The system shall reflect user changes within 5 minutes of submission.	MustHave
The website must allow Embedded Code.	MustHave	The website should log users in within 5 seconds.	MustHave
The website shall have a Search Function.	MustHave		
The website should have a Comments Section per blog post.	ShouldHave		
The website should feature Responsive Design to accommodate a variety of devices. MustHave.	MustHave		
The website must feature SEO Optimization	MustHave		

In our Development phase, we found the About-Me to be redundant for current iterations, given the short bio on our homepage. This iteration found the portfolio/project links pages to be a bit redundant given the initial scope and links to our qualifying GitHub and LinkedIn in our created Navbar. Some user deliberation found that even at current scope, the About-Me and

portfolio buttons/links could be consolidated into a more comprehensive separate page to be more visually appealing, informative, while maintaining the removal of redundancy. This page could store a more informative About-Me, as well as a formatted copy of our working resume, and some more specific cards to feature highlighted projects and interest! This idea seems more fitting than being separated components and sections, as well as it reduces the user's need to click around, which will be especially important for accessibility and screen-readers.

Initially, we intended the social media sharing buttons to link and pre-fill visitors posts on their respective platforms, enabling better discovery once our app is deployed. While this is still something future iterations hope to solve, more research is needed on React-Share as an npm project dependency. The TL:DR being that we should be able to utilize the React-Share library in order to import all the required iconography, as well as allow us to quickly pass props such as our URL and title to our share button components so that users can quickly share our posts as the website gains more traction (Larocca, 2024). As the website is still currently being worked on to improve functionality, namely routing via Slug instead of documentId for readability and memorability, this feature has been pushed for the time being until said underlying functionalities can be better implemented in our URLs.

While Strapi only seems to be able to directly pass on a documentId from our useParams field on our article page, it doesn't seem to be able to call the slug as a modifier in our GraphQL request without using a filter. Strapi's documentation mentions filters in its REST API documentation (Strapi, REST API: Filters), but it seems a bit lacking for current versions of Strapi. Some more research revealed our call may look similar to "http://localhost:1337/api/posts?filters[slug][$eq]='slug'" (Myddna, 2021); otherwise we'll have to modify Strapi Query Engine's 'routes' and 'controller' functions, which will require much

more research it seems depending on the implementation (Hamza, 2024) (Ovinz et al., 2023).

Similarly, the search function has been commented out until we can gain a better understanding of how Strapi processes queries via GraphQL, as well as all the possible ‘catch’ blocks we may need to implement in order to address any issues that might arise when allowing unpredictable user input. Normally, I would just add some transformations and requirements for input; but I want users to be able to input whatever they like into our search bar, querying broadly across our posts. Upon failure to find any meaningful relevancy from the users input, I still want the application to serve some of the websites recent articles to keep the user engaged. This seems to still be possible using Strapi’s Query Engine and filters, but maybe limits us to just a few filters for search. To implement better typo-tolerance, search filtering, semantic search, and predictive search, we may want to implement a plugin like Meilisearch for our final implementation once we’re able to route via slug. As said, more research is still needed for proper implementation, but Meilisearch have even provided some documentation regarding integrating with Strapi and they’ve included an importable tool called “instant-meilisearch” for our front-end to help render our search bar and filtering (Vermandel, 2021)! This approach will better ensure proper usability and resiliency in our search bar, so this seems to be the most optimal approach in regards to functionality and time-savings.

Since the beginning of this phase, essentially the beginning of our front-end development, we’ve stumbled across many different challenges and errors, many of which have now been briefly detailed in our development branch commits to our public GitHub.

GitHub: https://github.com/sweettigertea/VRTS-dev-blog

These setbacks have provided many exemplary testing and exciting learning opportunities throughout! In this time, 20 days or so, we’ve learned to completely render our two

main pages detailed in our Design phase while overcoming and learning to understand many of the errors and implementations we've come across! We stumbled across many console errors related to Vite's strict out-of-the-box controls, which we've opted to keep for learning purposes. We learned to use react-router-dom with Strapi API calls to render our pages. We learned more about TailwindCSS and it's various markdown capabilities and plugins (Smoljames, 2024). We learned to better manage our code base using Prettier's Tailwind plugin and how to apply it to our existing code (Reinink & Wathan, 2022). We ran into some left over dependency errors from Husky and Git when installing our BlocksRenderer, which we've learned to read and understand to clear up current and future conflicts. We learned to use BlocksRenderer to dynamically display our content from our rich text editor, which we can now further customize with Tailwind's markdown plugin to display dynamic content how we intend, and so much more (Strapi, Strapi Blocks React Renderer)!

In short, while we have the confidence and ability to add proper category sorting using GraphQL API calls and routing now, the time constraints as of now have deemed this implementation more appropriate for an added iteration. Additionally, with the lack of current articles on the site, sorting seems a bit unnecessary for usability, but is still immediately important to properly demonstrating the skills and expertise I've learned throughout my computer science program, so this should come first in our features backlog.

To implement category sorting, we can actually utilize the same methodology used to render our articles through our react-router-dom and using GraphQL relational API calls (Ninja, 2021). This provides a number of benefits. We can add better SEO for our categories as the website grows by utilizing a separate page with separate URLs for each category, making this feature more memorable and searchable; and this method will make optimizations for screen-

readers more clear while still being dynamic. Since screen-readers are essentially lists, this will create more surface level lists, instead of having to navigate back-and-forth through one big lists with dynamic sub-lists, reducing the amount of clicking and waiting around needed to interact with specific features. However, adjustments of our schedule may still allow this feature to be implemented before the projects deadline on May 4th!

Lastly, one of our requirements was SEO optimization, as this is websites purpose is to dynamically showcase ones portfolio and interest while networking with others. Again though, due to further research and development required to implement proper slug routing, as well as the security concerns that may come with deploying a project publicly, this requirement has also been pushed until after the Implementation and Deployment phase to reduce web traffic until security concerns can be properly assessed and managed. Additionally, after further research, Search Engine Optimization seems to a much larger topic than anticipated and can probably be reserved to own iteration by itself after some of the other requirements have been met.

With all that said, we were able to implement many of the features we laid out in our business requirements during our Analysis phase. We utilized Integration testing, End-to-End (E2E) testing, and finally User-Acceptance testing to validate our the remaining business requirements; the current scope of which includes our blog section with recent articles, responsive design, embedded code and image support, and user application performance.

Our integration testing was performed throughout the development process by using Apollo hooks to make GraphQL queries. We used console logs to manage where our calls were coming from and to ensure efficient data fetching was coming with our responses when rendering our content.

```
console.log("Homepage");
// console.log(ArticlesTest)
// console.log('gql')
console.log(data.articles);
// const articles=[
//     {
//         'id':1,
//         'title':'Article Title',
//         'desc':'Lorem ipsum dolor sit amet consectetur adipisicing elit. Suscipit, fugit fuga ab beat
//         'coverImg':imgPlaceholder,
//         'contents':'Lorem ipsum dolor sit amet consectetur adipisicing elit. Nulla distinctio libero
//         'authorName':"Author Name",
//         'authorImg':imgPlaceholder,
//         'authorDesc':'Author Description',
//     },
```

```
Homepage
                                                    Homepage.jsx:57:11
▼Array(7) [ {…}, {…}, {…}, {…}, {…}, {…}, {…} ]
  ▶0: Object { __typename: "Article", documentId:
  "bi8poiahxvzldxeqa2t1ab1n", slug: "beautiful-picture", …
  }
  ▶1: Object { __typename: "Article", documentId:
  "cgmh47io68quli7u7pezts2k", slug: "the-internet-s-own-
  boy", … }
  ▶2: Object { __typename: "Article", documentId:
  "co2k7w76eeu6iw1sst7teqm4", slug: "this-shrimp-is-
  awesome", … }
  ▶3: Object { __typename: "Article", documentId:
  "pe1nn8zhsly2krwp9j8dsn48", slug: "what-s-inside-a-black-
  hole", … }
  ▶4: Object { __typename: "Article", documentId:
  "hzr5zmg9sz5spplh5u7veyi9", slug: "a-bug-is-becoming-a-
  meme-on-the-internet", … }
  ▶5: Object { __typename: "Article", documentId:
  "gukq68l9subcd7i15eum7jt3", slug: "bingbong", … }
  ▶6: Object { __typename: "Article", documentId:
  "d1k94skrrkqla1y4d1355nf9", slug: "test", … }
   length: 7
```

```
Article Details
                                                    Article.jsx:62:11
▼Object { __typename: "Article", documentId:
"hzr5zmg9sz5spplh5u7veyi9", slug: "a-bug-is-becoming-a-
meme-on-the-internet", title: "A bug is becoming a meme on
the internet", description: "How a bug on MySQL is becoming
a meme on the internet", cover: {…}, author: {…}, content:
(6) […] }
    __typename: "Article"
  ▶author: Object { __typename: "Author", name: "Sarah
  Baker", email: "sarahbaker@strapi.io", … }
  ▶content: Array(6) [ {…}, {…}, {…}, … ]
  ▶cover: Object { __typename: "UploadFile", url: "/
  uploads/
  a_bug_is_becoming_a_meme_on_the_internet_6e8d10fb59.jpeg"
  }
    description: "How a bug on MySQL is becoming a meme on
    the internet"
    documentId: "hzr5zmg9sz5spplh5u7veyi9"
    slug: "a-bug-is-becoming-a-meme-on-the-internet"
    title: "A bug is becoming a meme on the internet"
```

Using GraphQL, we ensure we're getting only the data we ask for, which ensures more efficient data retrieval! We also used Firefox's built-in developer tools to test our applications load times under different throttling conditions. Under DSL conditions, which is still be pretty common around the world for loading web apps, the website took about 10 whole seconds to load with no caching enabled. Coincidentally, under the same conditions, GitHub takes about the same amount of time, but they load some of the assets ahead of time to make the experience feel less frustrating. This indicates that while our load time for the full application may actually be decently performant, there's more research to do in regards to making the site load more efficiently and make it more engaging while loading. Perhaps we should utilize more static

placeholders with loading animations before loading all our request. This may make the experience more bearable for users with slower internet connections, but may also add a bit of processing and call-time, so further research will need to be done concerning the methodologies of static loading in our stack, TailwindCSS animated icons and their benefits or drawbacks in regards to less performant clients, and the structuring of skeleton framing for our site to load more gradually. We could also add a spinner to our placeholder 'Loading…' to call the users attention when we're making our request to make it more apparent that is what we're waiting for.

Additionally, tying into our E2E testing, we utilized placeholder JSON test as well as various other integration test as we learned more about the various components of our chosen stack; many of which will be left commented in our development branch for preservation and continuous learning.

We made use of End-to-End (E2E) testing throughout the development of our application to ensure features behaved as expected as we implemented them. While other unexpected errors may occur during the projects lifetime, by simulating our users experience we can verify key components aren't being overstepped in regards to time-constraints and we can hone in when bugs occur, ruling out some potentials as we experiment and test. For the time being, we tested the current complete application flow from back-end to front-end. We created another Author, we created another entry, and tested leaving various values blank. This research led us to learn more about Strapi's 'required' fields in order to prevent null values and we've started further research into creating more engaging catch values for when important data calls may fail to respond.

We also tested various components of our BlocksRenderer in regards to embedded images and code blocks, in which we spent some more time researching markdown and how our

TailwindCSS-Typography plugin cycles through our rich text components in order to properly render our images and code blocks, more on the plugin later.

Lorem ipsum dolor sit amet consectetur adipiscing elit. Quisque faucibus ex sapien vitae pellentesque sem placerat. In id cursus mi pretium tellus duis convallis. Tempus leo eu aenean sed diam urna tempor. Pulvinar vivamus fringilla lacus nec metus bibendum egestas. Iaculis massa nisl malesuada lacinia integer nunc posuere. Ut hendrerit semper vel class aptent taciti sociosqu. Ad litora torquent per conubia nostra inceptos himenaeos.

```
<div className="prose max-w-3xl pt-6 wrap-anywhere indent-8">
<BlocksRenderer let content={article.content} />
</div>
```

Lorem ipsum dolor sit amet consectetur adipiscing elit. Quisque faucibus ex sapien vitae pellentesque sem placerat. In id cursus mi pretium tellus duis convallis. Tempus leo eu aenean sed diam urna tempor. Pulvinar vivamus fringilla lacus nec metus bibendum egestas. Iaculis massa nisl malesuada lacinia integer nunc posuere. Ut hendrerit semper vel class aptent taciti sociosqu. Ad litora torquent per conubia nostra inceptos himenaeos.

Lorem ipsum dolor sit amet consectetur adipiscing elit. Quisque faucibus ex sapien vitae pellentesque sem placerat. In id cursus mi pretium tellus duis convallis. Tempus leo eu aenean sed diam urna tempor. Pulvinar vivamus fringilla lacus nec metus bibendum egestas. Iaculis massa nisl malesuada lacinia integer nunc posuere. Ut hendrerit semper vel class aptent taciti sociosqu. Ad litora torquent per conubia nostra inceptos himenaeos.

We then spent some time navigating and testing various view ports and actions to ensure our changes scaled well during our development.

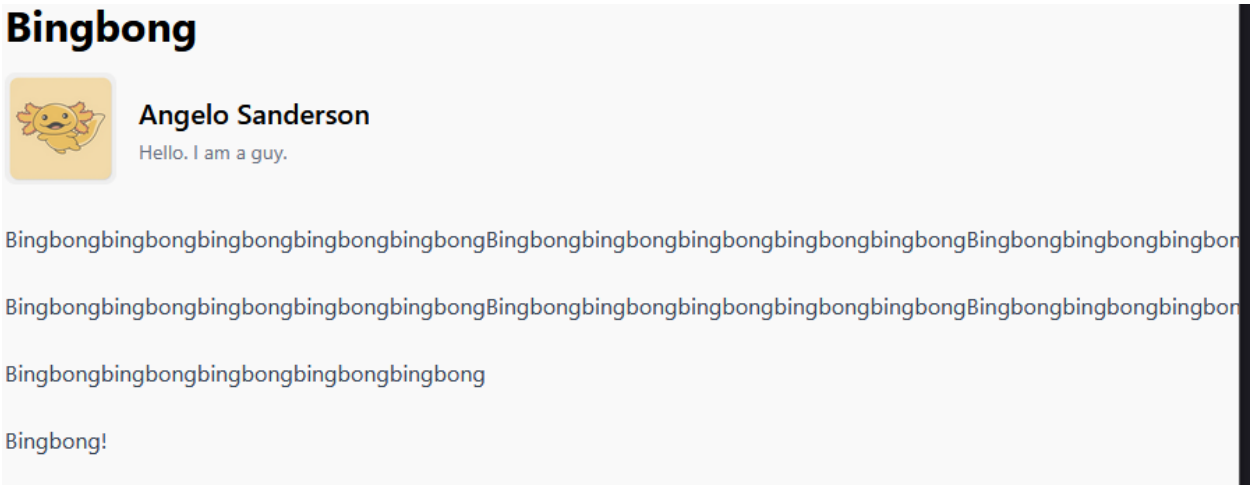
Finally, we utilized User Acceptance Testing, or UAT, to validate our own E2E testing in regards to responsive design, Login time, and publishing time. This test case involved logging into Strapi, navigating through content management to create an entry and author, inputting details, publishing the article, and checking functionalities such as responsive design, published posts displaying, standard prose rendering, and navigation via navbar.

sttDevBlog – End-2-End (E2E) User Acceptance Testing (UAT)

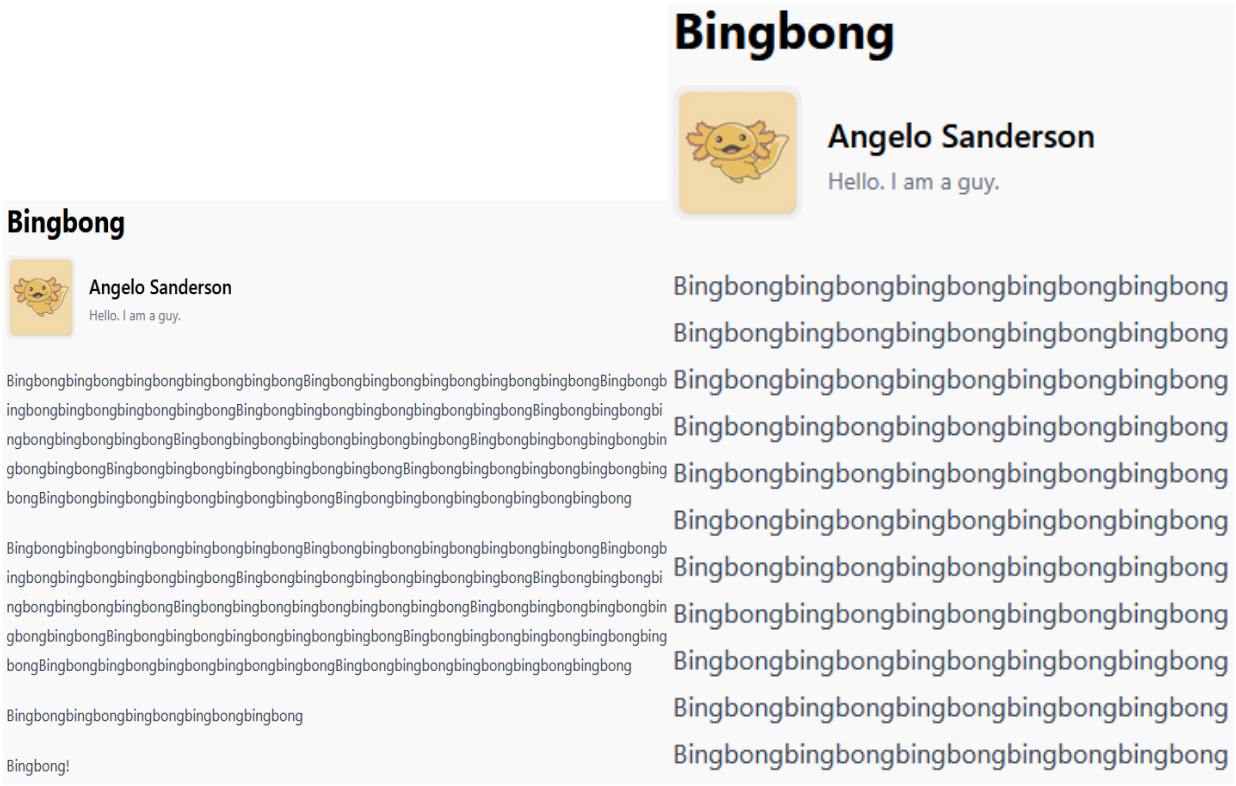
Release #	Week 11 – Testing Phase
Process	Writing a blog post – Strapi Backend
Test User	SweetTeesTea's DevBlog: Tyler Sanderson
Login Information	Link: http://localhost:1337 Company: sttDevBlog Username: xxx Password: xxx
Tester	Angelo Sanderson
Scenario Name:	SIDBP - 4.6.25
Test:	User creates blog post. Test user verifies post from front-end!
Script Description:	Our user will login into our Strapi portal and write a test blog post; then we will evaluate if said post shows up in our front-end per our business requirements.

Step #	Step	Data	Expected Outcome	Actual Results	Pass/Fail	Comments/Issues
1	Login to Strapi.	**Login Info**	Logs user into Strapi Back-end	Logged into Strapi Back-end	Pass	
2	Navigate to content-management > Author > Create new entry!		Allows user to create new entry!	User created a new entry	Pass	
3	Input name, description, email, and avatar.	**Avatar: Axolotl, or from upload...**	Creates new author!	User created new author.	Pass	
4	Navigate to Article > Create new entry!		Allows user to create new article!	User was able to access create new entry	Pass	
5	Input title, slug (generated), description, author, cover, categories, and content!		Allows user to create new Article. Generates slug automatically.	User was able to input article information, save, and publish.	Pass	Requires checkbox selection to select cover image. Need permissions for 'categories'...
6	Select Save, then Publish!		Publishes post to the front-end!	User successfully published post to the front-end	Pass	
7	Navigate to "http://localhost:5173/"		Displays new article in card format on homepage.	New article was displayed in card format on homepage	Pass	
8	Click on newly created article to view your magnificent work! :o		Displays article in standard prose!	Article text was displayed, but continuous text string did not wrap.	Fail	Text needs to wrap in case of continuous string of text
9	Click links in navbar.		Navigates browser to Home, Github, or Linkedin respectively.	Navbar links navigated appropriately.	Pass	
10	Do whatever you want! Try to break it!		Doesn't break!	Nothing broke!	Pass	

This test case actually revealed some interesting challenges! While our test passed in regards to responsive design in most cases, our tester actually found an egregious formatting error that forced our content outside the range of its container off of our view-port!



To fix this within our prose, we've added 'wrap-anywhere' to various elements across our app to account for continuous string of characters. TailwindCSS handles text wrapping by wrapping normal by default. This means that our content will dynamically adjust in order to render whole words until the view-port is too tiny, in which the text will break anywhere to wrap into the view-port, fixing our current issues with wrapping standard text within our prose!



This issue also presents itself in regard to our code block due to container issues. The best way to solve this problem so far seems to be by modifying our TailwindCSS Typography plugin in order to render our content more dynamically and to apply more standardized formatting depending on the specific element (Tailwindlabs), but more research and experimentation will need to be done on this topic in order to properly cycle through our 'prose-pre:prose-code' blocks in order to render our containers and their contents appropriately...

```
m dolor sit amet consectetur adipiscing elit.
apien vitae pellentesque sem placerat. In id c
s duis convallis. Tempus leo eu aenean sed di
nar vivamus fringilla lacus nec metus bibendu
nisl malesuada lacinia integer nunc posuere.
ass aptent taciti sociosqu. Ad litora torquent
os himenaeos.

Name="prose max-w-3xl pt-6 wrap-anywhere
derer let content={article.content} />
```

```
▼ <pre>
  ▼ <code>
      <div className="prose max-w-3xl pt-6 wrap-anywhere indent-8"> <BlocksRenderer let
      content={article.content} /> </div>
    </code>
  </pre>
```

Additionally, there are a few elements such as the Index (Adegbemile, 2024) (Strapi Table of Contents Guide 2025), word count, and read time (Paidly) for our articles that have been commented out as we continue to research and experiment with different methodologies of auto-rendering said elements. None of the implementations researched seem inherently difficult to implement, so these issues may be cleared up within this iterations development; however, since these elements aren't necessary to the overall fit and function of the current application, so they've been commented out for the time being until proper implementation has been reached.

In regards to the current overall user experience and functionality of our stack, the website was able to login, publish, and display our content within seconds of update, invalidating our current concerns in relation to performance and usability from our user's perspective!

In conclusion, many more challenges presented themselves as learning and testing opportunities. The testing phase successfully validated most of our required functionalities, with

minor issues requiring attention to enhance the overall user experience, system robustness, and showcase of our ability and excitement to learn more! We discussed our use of Integration testing, End-2-End testing, and User Acceptance Testing to measure our successes against our business requirements and discussed some of the challenges and measures that have been taken, or will be taken, in order to address these issues in regards to usability and reliability across various devices and user interactions. In our Implementation and Maintenance phase, we will continue to implement features and discuss the progress and research we've made in securing and hardening our application for public release on a Virtual Private Server!

Implementation & Maintenance Phase: Building a Blog with Vite + React, Strapi, and Tailwind CSS

The Implementation and Maintenance phases are crucial for ensuring a successful transition of a blog platform from development to operational use. These phases involve the technical setup and deployment, user adoption through proper documentation and training, and the establishment of a support system (help desk) to address issues promptly. By focusing on stability, user-friendly features, and continuous updates, these phases guarantee that the platform effectively serves its intended purpose, providing value to users by facilitating knowledge sharing and learning while maintaining robust performance and reliability over time.

The paper focuses on the challenges encountered during the transition from local hosting to VPS deployment, emphasizing security concerns and the integration of CI/CD pipelines; as well as discussing some of the key activities of distributing the system to users, such as creating user documentation, training, and building a user support system.

Before discussing distribution to users, it's important to discuss some of the set-backs we encountered during our technical set-up and deployment. Initially, we had planned to host our

website directly on our own infrastructure at home, but further research into best security practices when deploying publicly further dissuaded us from hosting locally (Dreams of Code, 2024), given the other services and data that is now hosted on our server. This unfortunately meant that we had to move VPS deployment up in our development backlog in order to ensure a secure environment for security testing and to post the project publicly as intended.

Deploying our project publicly presented a lot more security concerns than I had originally considered as well. To address some of these issues in our VPS set-up, hosted by Hostinger, We were able to set-up an Ubuntu LTS machine and hardened our system by setting up ssh key authorization with randomly generated pass code (Kurankat & Xiota, 2020). We also disabled root access and password ssh login, as well as set-up our Uncomplicated FireWall, or UFW, to only allow SSH access until other ports are opened and managed by our Nginx web server. We then installed Crowdsec to detect and block malicious IPs and spamming, which proved useful when checking our authorization log in attempts in the terminal (Dreams of Code, 2024)...

Research into deployment practices led us to research on the Continuous Integration & Continuous Delivery pipeline, or CI/CD, exposing key areas for our developer (me) to focus on . A good CI/CD ensures scalability, security, and redundancy in the case of failures (theSeniorDev, 2024). Specifically, our applications pipeline currently lacks redundancy in our planned deployment concerning a VPS, in the event of terminal disconnections or reboot failures; It lacks reliable Continuous Integration in that there must be downtime in order to migrate our database, or push a code change; it lacks automation in receiving coding changes safely, versioning, and pushing those changes to our deployed server; and more research is needed in order to better understand security best practices when moving a full-stack application

safely and securely without exposing private information. While there are many tools out there to achieve what we want, more research is needed to properly integrate these tools into our pipeline.

Tmux seems to solve the issue that we may face when developing on a VPS, such as ssh or terminal disconnections. By using Tmux, we can virtualize our terminal sessions independently, meaning we can detach can continue running them in the background (I9c, 2024). This is helpful for development, but doesn't really help keep our services running in the event of a failure, doesn't restart our services, and doesn't prevent downtime during system maintenance or migrations. We could use shell scripting and 'scp' to build and copy our application to our VPS (Tech With Tim, 2022), but that doesn't address our database migration and secret keys needed… It also doesn't address restarting the system, or rolling back the version in the event of a failure. While I'm sure these features could be solved with scripting and research, it doesn't seem optimal for scalability or time constraints.

With more research, we may be able to package our applications into a stack using Docker (PedroTech, 2024), then we could use Docker Swarm to take care of versioning and load balancing respectively (Docker Docs, 2024). This approach seems like it would take quite a bit more research to implement, but does seem to address most of our concerns related to versioning, load-balancing, and continuously integrating and delivering. Additionally, we could utilize GitHub Actions to implement most of the testing and evaluation methodologies we'd want to incorporate in our pipeline as the project grows (Strapi React GitHub Integration 2025), but more research would be needed here as well. There are self-hosted application options like Coolify that take care of a lot of the CI/CD pipeline, but even this would require more time and research in order to better understand all the necessary environment variables and components

needed in order to deploy the current projects database, secret keys, and access management on a public VPS. Due to time constraints, we spent some more time researching how we might deploy the project securely, manually…

While we have our project backed up and served locally, as well as we have version control on our code base using Git and GitHub, by default, we cannot – and should not – host our entire database and secret keys on our GitHub, especially publicly! Luckily it seems Strapi follows best practices and has marked secure details not to be included in our commits to GitHub. This was a relief, but also posed a challenge when cloning our repository over to the VPS. Since these details aren't included for safety and security, it also means we must learn to properly migrate our database to the VPS, as well as best practices when moving secure secret keys used for database management and authentication across our various back-end services (Strapi React GitHub Integration 2025)… So while we weren't able to get our existing database ported over to the VPS yet, we were able to port our Strapi project, content-sections, and APIs over to the new system. We may not port our database over since it's all for testing at the moment anyways and we don't want any secure info that has been used locally to be public on our VPS during devlopment, so we went ahead and just filled in an article, author, and category in the admin panel, after creating another secure user, in order to test our application on the VPS. However, we *will* want to understand this in the future, as migration of potentially private databases is a skill we need to learn in our career.

Changing our front-ends code to reflect the endpoint change also further exemplified our need to continue to refactor and simplify commonly used code into separated components, such as obfuscating our development IP address into an importable .env, environment variables (Strapi, Environment configuration and variables). This would make migrations and deployments

easier for those looking to clone the project for their own use as well.

While we were able to observe our functional application on the VPS on our VSCode server instance while running the services via npm, and we were able to open our firewall to view the Strapi Admin Dashboard externally via the IP and port number publically, we still can't seem to do the same with our Vite server via npm to view the front-end publically, meaning there is more we need to understand, in total, in order to serve the project on the VPS with Vite, Nginx, and our firewall. However, the time constraints for this assignment don't fit within the research time needed in order to increase our understanding enough to deploy the project publically via VPS either, so this research and testing will be implemented in further iterations until we are able to publicly deploy the project on our VPS.

Set-backs in our deployment, however, only minimally effect current distribution of user documentation, training, and support; since Strapi gives us plenty of tools to manage what our users can see and do when logged in through the back-end web portal. This means we can use user feedback from our continuous testing to implement changes to our Strapi web portal in order to make management easier upon full deployment.

Revisiting with our test case user, documented in our testing phase, we sat down to discuss key points where documentation may be needed to assist new users in order to get started quickly. Key points included expanding and formatting instructions for making key posts such as author, categories, and articles. Additionally, notes and further documentation should be included regarding content uploads and the 'content' rich text blocks editor and it's uses (Sanderson & Sanderson).

Luckily, our Strapi Admin panel makes it easy to create user logins, as well as restrict and limit what they're able to access and view in their respective accounts. This means we've

already been able to adjust our content flow and sections; and we can also restrict access to our content editor, so users only have access to creating content in the content manager. They don't have the ability to make any breaking changes to the back-end and helps direct users to what they do have access to, which is the content manager for creating Author, Categories, and Articles for the time being.

While this information will need to be expanded as we add functionality and features to the website, we were able to create some simple user documentation for navigating the current feature set.

Create an Author:

An author card is used for your attribution. Create one to represent yourself on articles!

- Select 'Content Manager' on the left bar.
- Select 'Author'.
- Select 'Create New Entry' at the top right.
- Input a screen name, email (optional), description, and any related articles you are associated with (if applicable).
- To add an 'avatar':
    - Select 'Click to add an asset or drag and drop one in this area'.
        - Use the checkbox to select an existing image.
        - Select 'Finish'.
    - To add a new image for 'avatar:
        - Click 'Add more assets' at the top right.
        - Click 'Browse Files' to upload from your computer, or upload from a URL.
        - Select 'Upload 1 asset to the library'.

        - Select ‘Finish’ in the bottom right corner.

- Once reviewed, select ‘Save’ in the top right corner to save your Author.

Create a Category:

Categories help separate articles by interest and topic. This is helpful for surfacing your articles to readers!

- Select ‘Content Manager’ on the left bar.
- Select ‘Category’.
    - Review the current categories and their associated articles.
- Select ‘Create new entry’ in the top right.
- Input a name (required), description, and select any associated articles (if applicable).
- To generate a slug (required), select the ‘⟳’ icon inside the box labeled ‘slug’.
    - Slug is critical for referencing your categories and for finding your categories in search engines. It’s appended to the website’s name in the URL for reference.
- Select ‘Save’ to save your new category!

Create an Article:

Articles are your main content!

- Select ‘Content Manager’ on the left bar.
- Select ‘Article’.
- Select ‘Create New Entry’ at the top right.
- Input title (required) and description.
- To generate a slug (required), select the ‘⟳’ icon inside the box labeled ‘slug’.
    - Slug is critical for referencing your articles and for finding your articles in search engines. It’s appended to the website’s name in the URL for reference (Patel, 2020).

- To add a ‘cover’ image:
    - Select ‘Click to add an asset or drag and drop one in this area’.
        - Use the checkbox to select an existing image.
        - Select ‘Finish’.
    - To add a new image for ‘cover’ image:
        - Click ‘Add more assets’ at the top right.
        - Click ‘Browse Files’ to upload from your computer, or upload from a URL.
        - Select ‘Upload 1 asset to the library’.
        - Select ‘Finish’ in the bottom right corner.
- Select ‘author’ to relate yourself and any other applicable authors with the article.
- Select related ‘categories’, if applicable.
- Input ‘content’

    Content is a Rich Text editor that allows you to write and manage your content.

    - In the editor, paragraphs behave like blocks of text. Hovering over a paragraph will display the ‘ ⠿ ’ icon, which can be clicked and dragged to reorganize the content.
    - Options to apply basic formatting, such as bold text, italics, underline, and strike-through can be found at the top of the editor.
    - Additionally, content can be enriched with code, links, images, and more in the scrollable drop-down located in top left of the editor (Strapi, Content Manager)!
- Select ‘Save’ in the top right, then select ‘Publish’ to publish the article to your website!

While the current iteration’s scope allows documentation to be easily distributed to our users using our local Nextcloud instance, we plan to include reference images and modify our Strapi back-end portal to include this documentation, to reduce the need to navigate away from

the application.

As for user support or help desk services, current iterations allow for direct communication between users and myself for rapid response and development, until a more robust system is iterated to. Should the project require, there are number of companies that offer scalable and flexible help desk solutions with great documentation and on-boarding, such as Zoho or Zendesk (Haan & Main, 2024); but current iterations don’t necessarily warrant the costs. Deployment to our VPS may warrant including a contact page within our Strapi back-end portal for support, but may not be necessary until more advanced user management features are iterated to.

In conclusion of our implementation and maintenance phase, while the project has exposed more scale than was initially planned, we were able to set-up, secure, and briefly deploy our project to a VPS; however, some more research will be needed to implement the wanted features securely, while positively contributing to our learning journey and CI/CD pipeline. Additionally, we were able to further evaluate our systems back-end and expanded on our user documentation, as well as considered further iterations in regards to user support as features are rolled out to this project.

References

(WAI), W. W. A. I. (n.d.). WCAG 2 at a glance. Web Accessibility Initiative (WAI). https://www.w3.org/WAI/standards-guidelines/wcag/glance/

3rd Wave Media. (2024, September 13). DevCard - Bootstrap 5 vcard & portfolio website template for software developers. UX Centred Bootstrap Templates & Themes For Developers. https://themes.3rdwavemedia.com/bootstrap-templates/resume/devcard-bootstrap-5-vcard-portfolio-template-for-software-developers/

Adegbemile, D. (2024, June 3). ow to create a Table Of Contents in React and Strapi rich text editor. Medium. https://medium.com/@adegbemile/how-to-create-a-table-of-contents-in-react-and-strapi-rich-text-editor-1a6ff7b4d10e

Amal. (2024, August 23). Vite & React: Alternative to create-react-app. Corbado. https://www.corbado.com/blog/vite-react

Austin, S. (2025, January 16). How to Determine the Ideal Size of a Web Server for Your Website. Marketing Scoop. https://www.marketingscoop.com/website/wordpress/how-to-determine-the-ideal-size-of-a-web-server-for-your-website/

Axe: Accessibility Testing Tools and software. Deque. (2024, June 3). https://www.deque.com/axe/

Axios API. Axios API | Axios Docs. (n.d.). https://axios-http.com/docs/api_intro

Bajgain, D. (2023, October 20). Creating a Dynamic Blog with Vite React, Strapi CMS, and Tailwind CSS. Darshan’s Blog. https://blogdarshan.netlify.app/blog/2

C., V. (2022, August 25). How much bandwidth and disk space you need for a typical blog website. FlipFall Magazine. https://flipfall.com/average-disk-space-bandwidth-website-need-hosting-plan/

Daemes, & STA. (2024, September 20). Message “support for password authentication was removed.” Stack Overflow. https://stackoverflow.com/questions/68775869/message-support-for-password-authentication-was-removed

Daniel. (2023, August 27). Free open-source blog template for developers. DEV Community. https://dev.to/danielcgilibert/free-open-source-blog-template-for-developers-218e

Dreams of Code. (2024, September 6). RO Setting up a production ready VPS is a lot easier than I thought. YouTube. https://www.youtube.com/watch?v=F-9KWQByeU0&t=705s

Finn, T., & Downie, A. (2024, December 19). What is a content management system (CMS)?. IBM. https://www.ibm.com/think/topics/content-management-system

GitHub, Inc. (2020, March 12). Git Guides - Install Git. GitHub. https://github.com/git-guides/install-git#debianubuntu

Haan, K. (2024, September 29). Best Help Desk Software. Forbes. https://www.forbes.com/advisor/business/software/best-help-desk-software/

Hamza, M. (2024, November 23). How to search by slug instead of ID in Strapi V4. Medium. https://medium.com/@malikhamzav/how-to-search-by-slug-instead-of-id-in-strapi-v4-2a66e24f0dab

Hover, Focus, & Other States. Tailwind CSS. (n.d.-a). https://v2.tailwindcss.com/docs/hover-

focus-and-other-states

I9c. (2024). How to Install and Use Tmux. How to install and use tmux. https://tmuxcheatsheet.com/how-to-install-tmux/

Introduction to lighthouse. Chrome for Developers. (2016, September 27). https://developer.chrome.com/docs/lighthouse/overview

James. (2024, December 24). Server administration: Understanding server load on linux. IT Crash Course - Essential IT Knowledge. https://itcrashcourse.com/server-administration-understanding-server-load-on-linux/

Kurankat, & Xiota. (2020, December 2). Copying SSH Key From Root To Another User On Same Machine. Ask Ubuntu. https://askubuntu.com/questions/1218023/copying-ssh-key-from-root-to-another-user-on-same-machine

Larocca, M. J. (2024, July 29). Implementing Dynamic Social Media Sharing in Your React App with React Share. SelfTaughtTXG. https://selftaughttxg.com/2024/07-24/implementing-dynamic-social-media-sharing-in-your-react-app-with-react-share/

Limosyn. (2021, April 29). An Exhaustive Guide for your Website’s Tech Stack. An exhaustive guide for your website’s Tech Stack. https://www.limosyn.com/posts/exhaustive-guide-for-website-tech-stack

Linuxize. (2020, June 1). How to Enable SSH on Ubuntu 20.04. https://linuxize.com/post/how-to-enable-ssh-on-ubuntu-20-04/

Main, K., & Aditham, K. (2025, February 4). Best blogging platforms. Forbes.

https://www.forbes.com/advisor/business/software/best-blogging-platforms/

Microsoft. (2025, March 3). VS Code Remote Development. Visual Studio Code. https://code.visualstudio.com/docs/remote/remote-overview

Mirchevski, B. (n.d.). How to determine the correct size and type of a web server. HostAdvice. https://hostadvice.com/how-to/web-hosting/how-to-determine-the-correct-size-and-type-of-a-web-server/

Myddna. (2021, December). Strapi V4 search by slug instead ID. Strapi Community Forum. https://forum.strapi.io/t/strapi-v4-search-by-slug-instead-id/13469/6

Navigating. React Router. (n.d.). https://reactrouter.com/start/library/navigating

Net Ninja. (2021, July 23). Strapi Crash Course (with React & GraphQL) #9 - Apollo Client Setup. YouTube. https://www.youtube.com/watch?v=Y5iDq-suzCA&list=PL4cUxeGkcC9h6OY8_8Oq6JerWqsKdAPxn&index=10

Ninja, N. (2021, July 28). Strapi Crash Course (with React & GraphQL) #13 - Fetching Related Data. YouTube. https://www.youtube.com/watch?v=Ym-UBtxril4&list=PL4cUxeGkcC9h6OY8_8Oq6JerWqsKdAPxn&index=14

Ovinz, Oikon, A., & Hagel, T. (2023, April 3). How to Fetch Strapi by slug, and how to populate the categories. Stack Overflow. https://stackoverflow.com/questions/73261870/how-to-fetch-strapi-by-slug-and-how-to-populate-the-categories

Paidly. (n.d.). Strapi Plugin Reading Time. GitHub. https://github.com/Paidly/strapi-plugin-reading-time

Palas, P. (2017, July 2). How I built a CMS, and why you shouldn't. HackerNoon. https://hackernoon.com/how-i-built-a-cms-and-why-you-shouldnt-daff6042413a

Patel, R. (2020, August 7). Wordpress Slug: What it is and why it matters. Tribulant Blog. https://tribulant.com/blog/wordpress/wordpress-slug-what-it-is-and-why-it-matters/

PedroTech. (2024, March 20). RO Deploying A React App To Docker | Become A React Pro. YouTube. https://www.youtube.com/watch?v=RL31Kv_PHTw

Postman. (2025, March 26). Postman Docs. https://learning.postman.com/

prachipaliwal1207. (2024, September 23). What is agile methodology?. GeeksforGeeks. https://www.geeksforgeeks.org/what-is-agile-methodology/

prachipaliwal1207. (2024, September 23). What is agile methodology?. GeeksforGeeks. https://www.geeksforgeeks.org/what-is-agile-methodology/

pwizla. (n.d.). Content API: Strapi 5 documentation. Content API | Strapi 5 Documentation. https://docs.strapi.io/dev-docs/api/content-api

Rapidly build modern websites without ever leaving your HTML. Tailwind CSS. (n.d.-b). https://tailwindcss.com/

Reinders, M. (2021, November 12). The Ultimate Guide to Installing OpenSSH on Windows. Petri IT Knowledgebase. https://petri.com/the-ultimate-guide-to-installing-openssh-on-windows/

Reinink, J., & Wathan, A. (2022, January 24). Automatic Class Sorting with Prettier. TailwindCSS. https://tailwindcss.com/blog/automatic-class-sorting-with-prettier

Sanderson, T., & Family Member, R. (2025, February 13). Professional Blog Requirements? personal.

Sanderson, T., & Partner, R. (2025, February 13). Professional Blog Requirements? personal.

Sanderson, T., & Sanderson. (2025, May 1). Test Case Revisit. personal.

Sanderson, T., & Superior, R. (2025, February 13). Professional Blog Requirements? personal.

Smoljames. (2024, March 14). Learn TailwindCSS in 23 minutes. YouTube. https://www.youtube.com/watch?v=W-LDhPyv478

Stapi.io. (n.d.-a). Build modern websites with the most customizable Headless CMS. Strapi. https://strapi.io/

Strapi. (n.d.). Content Manager. Strapi 5 Documentation. https://docs.strapi.io/cms/features/content-manager

Strapi. (n.d.). Environment configuration and variables. Strapi 5 Documentation. https://docs.strapi.io/cms/configurations/environment

Strapi. (n.d.). Strapi CMS for content teams. https://strapi.io/for-content-teams

Strapi. (n.d.-a). REST API: Filters. Strapi 5 Documentation. https://docs.strapi.io/cms/api/rest/filters

Strapi. (n.d.-b). Graphql API. Strapi 5 Documentation. https://docs.strapi.io/dev-docs/api/graphql

Strapi. (n.d.-b). Strapi Blocks React Renderer. GitHub. https://github.com/strapi/blocks-react-renderer

Strapi. (n.d.-c). strapi / license. GitHub. https://github.com/strapi/strapi/blob/main/LICENSE

Strapi.io. (n.d.). Strapi Docs - Installing from CLI. CLI | Strapi 5 Documentation. https://docs.strapi.io/cms/installation/cli

Strapi (Ed.). (n.d.). The best headless CMS FOR REACT. The best Headless CMS for React. https://strapi.io/integrations/react-cms

Strapi React GitHub Integration. Restackio. (2025, March 25). https://www.restack.io/p/strapi-knowledge-strapi-react-github-integration

Strapi Table of Contents Guide. Restackio. (2025, April 18). https://www.restack.io/p/strapi-knowledge-strapi-table-of-contents-guide

Swarm mode key concepts. Docker Documentation. (2024, September 10). https://docs.docker.com/engine/swarm/key-concepts/

Tailwind (Ed.). (n.d.). Rapidly build modern websites without ever leaving your HTML. Tailwind CSS. https://tailwindcss.com/

Tailwindlabs. (n.d.). Tailwind CSS Typography. GitHub. https://github.com/tailwindlabs/tailwindcss-typography

TechHut. (n.d.). TechHut.tv. https://techhut.tv/

Tech With Tim. (2022, April 2). How To Deploy A React App - Using NGINX & Linux. YouTube. https://www.youtube.com/watch?v=KFwFDZpEzXY

theSeniorDev. (2024, August 15). how to deliver end-to-end like a Senior Dev. YouTube.

https://www.youtube.com/watch?v=nRVDjqojnNE

Vermandel, C. (2021, May 4). How to add relevant search to your Strapi app using Meilisearch. Meilisearch. https://www.meilisearch.com/blog/add-meilisearch-to-your-strapi-app

W3Schools. (n.d.). Git Getting Started. W3Schools Online Web Tutorials. https://www.w3schools.com/git/git_getstarted.asp?remote=github

Web Dev Simplified. (2022a, July 7). Junior vs Senior React Folder Structure - How To Organize React Projects. YouTube. https://www.youtube.com/watch?v=UUga4-z7b6s

Web Dev Simplified. (2022b, August 2). Learn React Router v6 In 45 Minutes. YouTube. https://www.youtube.com/watch?v=Ul3y1LXxzdU&t=1301s

Web Dev Simplified. (2023, April 8). Learn React With This One Project. YouTube. https://www.youtube.com/watch?v=Rh3tobg7hEo&t=37s

Why vite. vitejs. (n.d.). https://vite.dev/guide/why.html